



ALWAYS ON

Field	Value
Document	Complete single-file architecture, integration, security, operations, evidence, and active-work report
License	CC BY-NC-SA — creativecommons.org
Project origin	Building ~2010 · Drone ~2012 · Linux systems ~2023
Website	https://www.300x3.com
Supporting plan	https://archive.org/details/@scott_widmann
Created with	Bluebeam and LibreDraw (PDF project plan), Perplexity.ai, Cline.bot (Markdown)
Revision	2026-09-29
Current main host	ATX desktop running Linux Kubuntu
Peripheral host	Drone — Raspberry Pi 5 and Autopilot Module, running KaliOS
Format rule	Tables and topology diagrams are primary; original detailed commands/evidence are retained in-place below for operational completeness
Reading order	ES.1 (Current Architecture Corrections) is the single source of truth for current architecture. Any statement elsewhere in this document that conflicts with ES.1 is superseded . ES.1-ES.3 are the current-state summary; sections 1-21 are the authoritative detailed record; where they disagree, ES.1 governs. Deviations are recorded in section 18 with rationale and compensating controls. A recorded deviation never silently overrides ES.1 — an open deviation means ES.1 describes the target and section 18 describes what is actually deployed.

Contents

Section	Title	Page
ES	Executive Summary	3
ES.1	Current Architecture Corrections	3
ES.2	Master Topology	4
ES.3	Implementation Status and Current Work	9
ES.4	Detailed System Record	12
1	System Purpose	13
2	Platform Baseline	14
3	High-Level Architecture	16
4	Security, Isolation, and Data Policy	21
5	Network Domains and Controlled External Access	24
6	Component Boundaries, GUI Reporting Tools, and Operator Access	29
7	Public Storefront and Payment Policy	31
8	Mapping and Photogrammetry	36
9	Field and LoRa Architecture	41
10	Simulation Architecture	43
11	Ledger, Provenance, Archive, and IPFS	48
12	Host Installation and Configuration	54
13	Podman Runtime and Quadlet Policy	57
14	Secrets, Service Identity, and Version Controls	60
15	Sales, Mastodon, OpenClaw, and Local AI	63
16	Scripts, Operational Standards, and Logs-Journals	68
17	Backup, Restore, Monitoring, and Completion Criteria	70
18	Approved Deviations and Open Decisions	72
19	Open Implementation Items	74
20	Current Verification Evidence	77
21	Status References	80
Appendix A	Revision History	81

Executive Summary

ES.1 Current Architecture Corrections

Subject	Current plan; replaces any contrary older text below
OpenClaw	Public marketing/contact across website chat, email, Mastodon, approved social channels; direct Mastodon publisher; local output is standardized PDFs for order/follow-up/support/payment workflows, which are then processed into the sales, payment, and ledger records
LM Studio	Fundamental local LLM host for OpenClaw
Browser	Konqueror — dedicated browser for automation; Tokodon removed completely
RNode client	MeshChatX exclusively, and provides the Reticulum network stack controls. End-to-end encryption is built into the stack: all communication is secured with strong modern encryption by default, all encryption keys are ephemeral, forward secrecy applies by default, and it is not possible to establish unencrypted links or send unencrypted packets
Radios	PEOPLE-RADIO = public human chat; DRONE-RADIO = authenticated private drone mission/status traffic; a local switch connects the fabrication equipment, a local router connects the IoT devices, and Wi-Fi/Ethernet from the main desktop reaches the main internet; the main desktop manages all DHCP
QGroundControl	Desktop primary mission planning; KaliOS RPi5 fallback/out-of-range mission-update operation; and drone — KaliOS on the Raspberry Pi 5 with the Autopilot Module, running ArduPilot
Prometheus	Prometheus is for security only. It acts alone and independently, operating on the other systems to ensure security and to address any problems: time-series store, rule evaluation, alerting, and security evidence
Grafana	Dashboards and metrics only; it reads the databases that already exist and does not write to them
Metabase	Reporting only, and it does ad-hoc read-only reporting, so it requires its own dedicated PostgreSQL application database to hold its Metabase schema, saved questions, dashboards, and subscriptions. That application database holds Metabase's own state only — it is not a system of record for business data, and it never receives data from the reporting sources. Metabase connects to the other PostgreSQL and MySQL databases, and to local desktop-application SQLite files, as a read-only user in order to report on them, and it never writes to them. Ad-hoc reports that become recurring are promoted into stable Grafana dashboards
Corda	Blockchain-enabled accounting, ledger, receipt, entitlement, fulfillment, provenance, and approved state-transition system. Built on Corda 5 with PostgreSQL. The previous V4 test installation and its database are removed, and no data needs to be migrated
Corda persistence	PostgreSQL. Corda is built on V5 against cordadb; the previous V4 test installation and its H2 database were removed 2026-09-28 and no data needs to be migrated. cordadb is a separate logical database on the host PostgreSQL 18 cluster with its own roles and backup scope, which is correct
IPFS	Post-sale marking/transfer only for approved map/imagery and telemetry/product-operational packages; no Corda/ledger/accounting/archive dependency
Backup/archive	Local authority + encrypted restic + encrypted pCloud replication; independent of IPFS and Corda
Secret authority	KDE Wallet; services needing Wallet secrets start after KDE login
GPU policy	Priority: desktop/Konqueror → SketchUp/SketchUp Web → active LM Studio/OpenClaw → ROS/Gazebo/SITL → WebODM batch
Additive fabrication	Real machines, not simulated cells: MainsailOS/Moonraker/Klipper on the BigTreeTech CB1/RPi, individual 3D printers and CNC machines; no OrcaSlicer reference. ao-sim-fabrication coordinates all industrial engineering and production related details, receiving specific production data from each machine, and runs the kitchen (§10.2)
CNC	No bCNC or current CNC software claim
Simulation (both domains)	ao-sim-vehicle and ao-sim-fabrication both require, as a baseline capability and not as optional extras: 3D world setup, boning, reinforcement learning objects , and an HTML portal to operation . Each domain stands up its own 3D world, builds its own boning/alignment and datum structure into that world, supplies reinforcement-learning objects as the trainable entities for its scenario and policy work, and exposes the whole environment through a browser-served HTML portal so the world can be set up and operated without a desktop GUI client. Details are in §10.1 (vehicle) and §10.2 (fabrication and facility)
Home automation	Domoticz/RPi for the usual Domoticz home automation features: HVAC, doors/locks/access, security/alarms, lighting/scenes, cameras, weather, environmental sensing, media, and other typical Domoticz device classes; separate from printers and future building robotics

ES.1.1 Change Summary

This is the **single copy** of the change summary for this revision. It is not repeated in §3, in Appendix A, or anywhere else in the document.

Change	Effect on this document
Login-gated secret delivery is intended	Services consuming KDE Wallet secrets start after Plasma login; the ~60s wait is the bounded startup allowance, not a fallback (§14.1.1)
No separate service account	All services run under the operator account; the alwayson-sales (UID 993) Mastodon placement is legacy history (§20.0)
KDE Wallet remains the secret authority	The org.kde.kwalletd6 bus and method names are flagged for host verification only (§14.1.1)
Corda uses PostgreSQL	Corda is built on V5 against cordadb on host PostgreSQL 18; the V4 test install and its H2 database were removed 2026-09-28 and no data is migrated (§18.2)
Metabase works	It runs on the host and serves its login page; the earlier "not serving" finding was a different, undeployed container
RF interference may simply be interference	WORK 000700 closes on a recorded characterization, not a fix
Consolidated topology	ES.2 is the single authoritative master diagram; §3.1 is a detail view of it, drawn from the same graphic rather than as separate ASCII
Implementation status consolidated	All implementation statuses are tracked once, as the single list in ES.3 . The former §3.2 status table is deleted, and the status columns in §19 and §20 now reference ES.3 IDs instead of restating a status
Work and issue items consolidated	Section 19 is one table of remaining items, each naming the standard it serves
Simulation renders relocated	The three Gazebo model views sit in §10.2 with captions
Appendix A reduced	Appendix A is the revision index only; it carries no change summary

The open work items are carried as a running list. They are recorded once, in ES.3 Current Work and §19 Open Implementation Items, and are deliberately not repeated here.

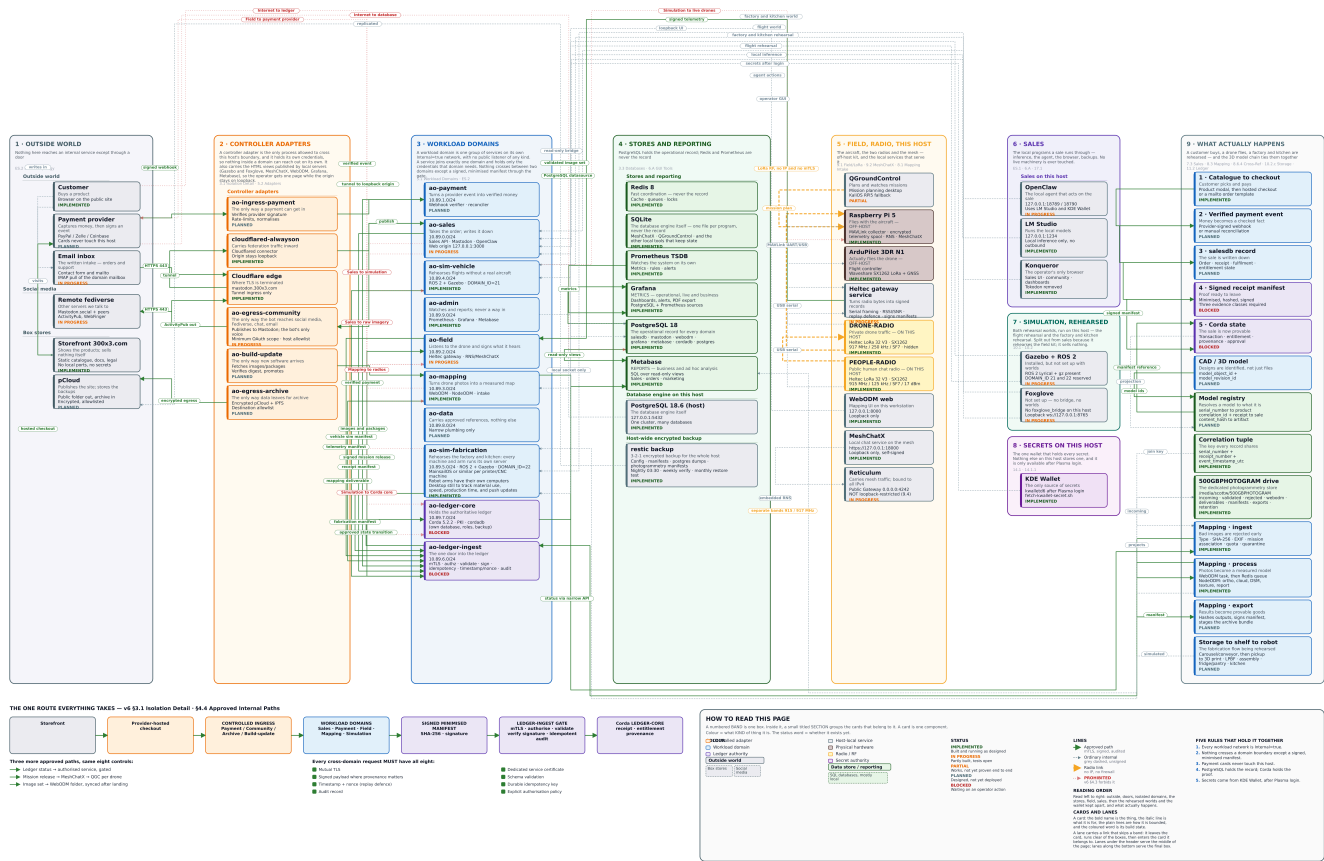
ES.2 Master Topology

This is the **single authoritative diagram of the entire project**. Every other diagram in this document is a detail view of one part of it and must not contradict it. Domain names, network names, and paths are abbreviated here and defined in full in the sections referenced.

ALWAYS ON — ONE-PAGE TOPOLOGY

What every part of the system is for — the outside world, the only data, the isolated domains, the single ledger gate, the stores

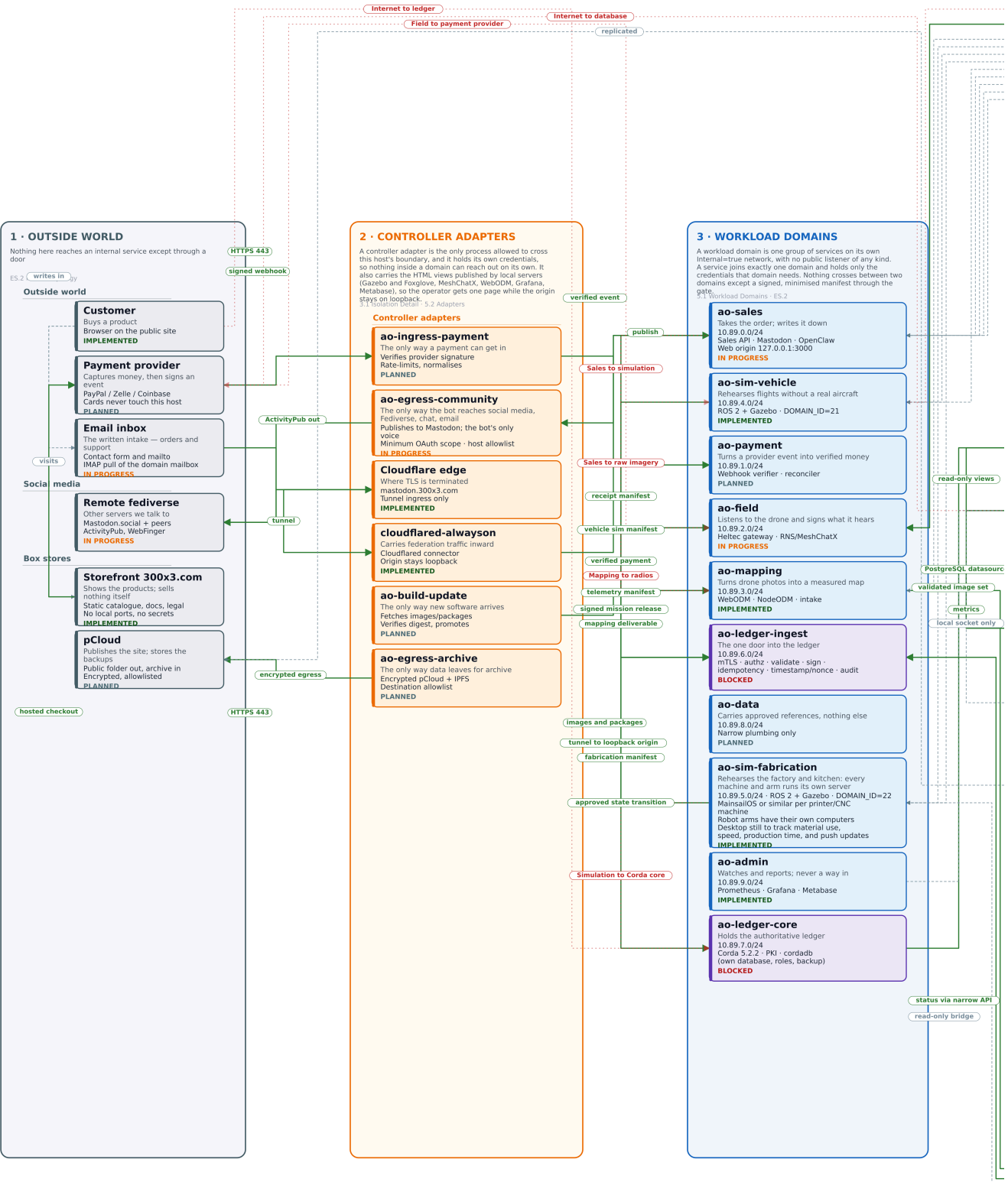
Nodes are the EXPECTED-to-BE-INSTALLED system, a PLANNED or BUDGETED node is design, not live state. Authority: v4 §3.2.1 §3.3.1 §3.10.2



ALWAYS ON — ONE-PAGE TOPOLOGY

What every part of the system is for — the outside world, the only doors, the isolated domains, the single ledger gate, the stores

Nodes are the EXPECTED-TO-BE-INSTALLED system. A PLANNED or BLOCKED node is design, not live state. Authority: v6 \$3.2 / \$3.3 / \$5.1 / \$5.2.



THE ONE ROUTE EVERYTHING TAKES — v6 \$3.1 Isolation Detail · \$4.4 Approved Internal Paths

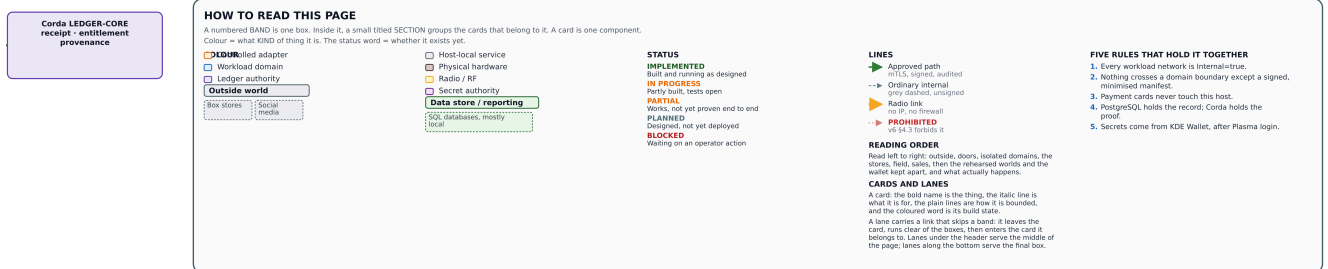
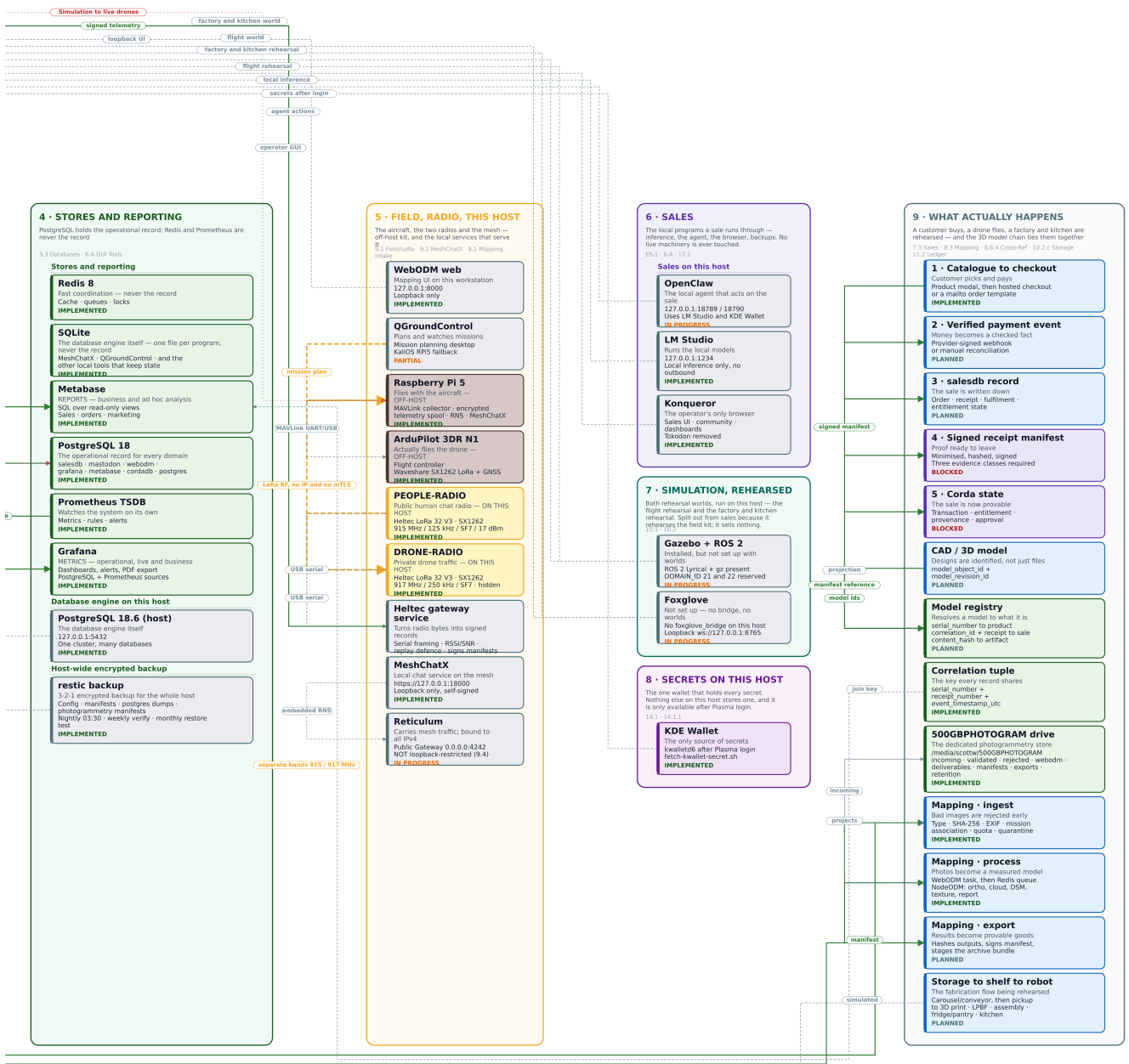


Three more approved paths, same eight controls:

- Ledger status → authorised service, gated
- Mission release → MeshChatX → QGC per drone
- Image set → WebODM folder, synced after landing

Every cross-domain request MUST have all eight:

- Mutual TLS
- Signed payload where provenance matters
- Timestamp + nonce (replay defence)
- Audit record
- Dedicated service certificate
- Schema validation
- Durable idempotency key
- Explicit authorisation policy



The only public entries. Nothing reaches an internal service directly. There are exactly four, each purpose-built and each carrying nothing but its own approved traffic:

Entry	Network	What it carries	Direction and status
Storefront — 300x3.com / www.300x3.com, static HTML in the pCloud Public Folder	pCloud (not a Podman network)	Products, docs, legal, downloads. Also the hosted-checkout origin and the public PDF intake forms.	Outbound publication; static asset delivery
Federation and chat — mastodon.300x3.com, and chat.300x3.com (OpenClaw relay / sitebot)	ao-sales via the Cloudflare Tunnel	ao-sales coordinates social media, email, and Mastodon; the AI bot and chat; and the order-request and receipt workflows. Mastodon web/streaming and OpenClaw chat. Origin stays loopback (127.0.0.1:3000, :4000, :18790). It is the only domain that touches customers directly.	Inbound via cloudflared-alwayson.service; outbound federation via the sales egress bridge
ao-ingress-payment	adapter	Zelle, PayPal, and Coinbase payment verification. Receives provider webhook/relay events, verifies the signature, and emits a normalized payment event.	Inbound. Planned — not yet deployed , blocked on the provider decision (\$18.4)
ao-egress-archive	adapter	Moving large data and the image/map/telemetry files into IPFS for transfer after sale , plus encrypted pCloud replication. Destination allowlist, separate credentials, transfer audit.	Outbound. Planned — not yet deployed , archive credentials pending
ao-build-update	adapter	Software updates. Image and package acquisition before controlled promotion, with digest capture and update audit. Never attaches to a workload.	Outbound. Planned — not yet deployed

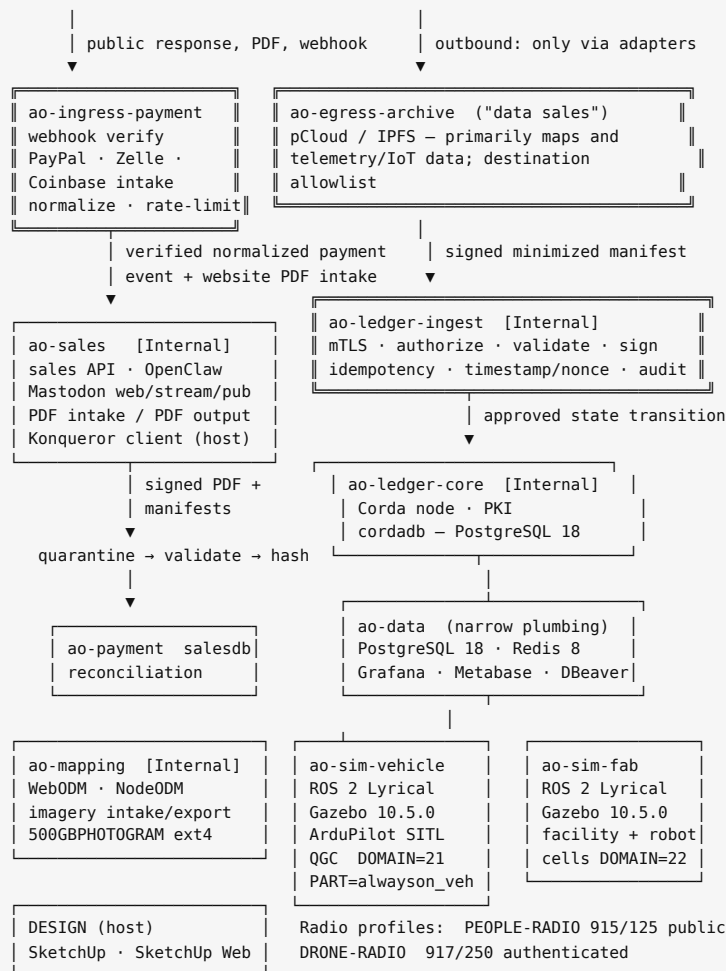
AO- means "ALWAYS ON". Every ao-* network is one isolation domain: the ten internal workloads (ao-sales, ao-payment, ao-field, ao-mapping, ao-sim-vehicle, ao-sim-fabrication, ao-ledger-ingest, ao-ledger-core, ao-data, ao-admin) are all Internal=true with no public listener, and the three adapters above are the deliberate exceptions that are allowed to reach the internet.

Question	Answer
Where does money move?	Hosted checkout at the provider → ao-ingress-payment → verified event → ao-payment/salesdb → signed manifest → ao-ledger-ingest → Corda. Cards are never handled locally.
Where else does a transaction enter?	From the website: email → PDF → Corda processing. A customer email produces a standardized PDF request which is processed into the ledger workflow.
Where is the ledger (the only copy)?	ao-ledger-core on cordadb in PostgreSQL 18, a separate database with its own roles and backup scope. PostgreSQL is authoritative (\$18.2).
What can the internet never reach?	Every ao-* workload network. All are Internal=true with no public listener.
What crosses a domain boundary?	Only a signed, minimized manifest through ao-ledger-ingest, under mTLS with authorization, replay defence, idempotency, and audit.
Where do secrets come from?	KDE Wallet, after Plasma login, by design. See §14.1.1.
Does monitoring need Grafana?	No. Prometheus is for security only; it acts alone and independently on the other systems to ensure security and to address any problems. Grafana is for stable dashboards and metrics; Metabase is for ad-hoc reporting by users, and a recurring ad-hoc report is promoted into a stable Grafana dashboard. Both read the databases that already exist.

The remaining text form of the topology, for readers who cannot view the diagram:

EXTERNAL (no route to any internal service below except via an adapter)

storefront 300x3.com – static pCloud Public Folder: products, docs, legal
Mastodon mastodon.300x3.com – federated via dedicated Cloudflare Tunnel
payment provider (PayPal / Zelle / Coinbase stablecoin) – hosted checkout
website email > PDF > Corda processing
pCloud archive (encrypted egress)



The fabrication machines – MainsailOS, Moonraker, Klipper, the individual additive-manufacturing machines (3D printers and CNC), and the BigTreeTech CB1 – are real machines, not simulation nodes, so they are NOT drawn as `ao-sim-fabrication` children. ao-sim-fabrication coordinates all industrial engineering and production related details, receiving specific production data from each machine, and it runs the kitchen. See ES.1 and §10.2.

Post-sale transfer (authorized recipient only – no Corda/archive dependency)

approved map/imagery or telemetry package → package · hash · recipient
authorization → IPFS transfer (private swarm / pinning / encryption)

SECRETS – KDE Wallet is the authority (ES.1)

Plasma login → kwalletd6 unlock → fetch-kwallet-secret.sh (ExecStartPre, ≤60s)
→ 0600 env file per unit. Services needing Wallet secrets start after login
BY DESIGN. All services run under the operator account; no separate service user.

MONITORING – Prometheus is for security only; Grafana is dashboards/metrics;
Metabase is reporting. Each tool keeps its own application database
and reads the business databases read-only; none of them is the
system of record.

host + services + exporters → Prometheus TSDB · rules · alerts ← acts alone
└─ Grafana (dashboards/metrics)
ao-admin carries Prometheus, node_exporter, Grafana, Metabase, backup/restore

ES.3 Implementation Status and Current Work

ES.3 is the one place in this document where a component is given an implementation status. There are no per-category status tables anywhere else: the former §3.2 status table has been deleted, and the status columns in §19 and §20 now carry an ES.3 status ID rather than restating a status. One component, one status, one place to change it.

ES.3.1 Implementation Status — the single list

ID	Component	Status	Current state	Next action
ST-01	Host platform — Ubuntu, Podman, Quadlet, protected administration	Implemented	Host inventory and base platform verified	Maintain the version matrix
ST-02	Domain isolation — ten internal workload networks	Implemented	Isolation test verified; all networks <code>Internal=true</code>	Add narrow adapters only as required
ST-03	Mapping — WebODM and the photogrammetry drive	Implemented with deviation	GPU-enabled smoke test completed, orthophoto produced; stack is rootless (scottw/mapping store)	Formalize the steady-state rootless/system model designation
ST-04	Field, Reticulum, and LoRa — RPi5, Waveshare LoRa, Heltec V3, MeshChatX	In progress	Both Heltec LoRa 32 V3/SX1262 RNodes functional and initialized by MeshChatX; PEOPLE-RADIO 915 MHz/125 kHz, DRONE-RADIO 917 MHz/250 kHz; 32 interfaces configured, none explicitly disabled; RF feedback observable on both bands	Measure and classify the feedback; record RSSI/SNR, noise floor, packet loss, airtime, retries, cross-band isolation (WORK 000700)
ST-05	Reticulum runtime and connectivity	Partial	Startup logs show auto-connections, peering, announces, and LXMf/Nomad announcements, but also timeouts, network-unreachable errors, and connection refusals; 29 TCP clients enabled	Characterize the connection failures; confirm the public-gateway exposure posture
ST-06	MeshChatX version provenance	Complete with verification pending	Desktop metadata declares 4.9.1; executable hash matches the local manifest	Confirm the running-version check against the declared version
ST-07	Vehicle simulation — <code>ao-sim-vehicle</code>	Implemented (headless runtime)	Headless Gazebo 300-iteration and ROS-Gazebo bridge tests passed; ArduPilot SITL HEARTBEAT validated over MAVLink	Build the baseline capability required by ES.1: 3D world setup, boning, reinforcement learning objects, and an HTML portal to operation
ST-08	Fabrication and facility simulation — <code>ao-sim-fabrication</code>	Implemented (headless runtime)	Headless Gazebo 300-iteration and bridge test passed; model views rendered in \$10.2	Build the same baseline capability required by ES.1: 3D world setup, boning, reinforcement learning objects, and an HTML portal to operation
ST-09	Ledger core — Corda on <code>cordadb</code>	Blocked	Corda 5.2.2 scaffolded on V5 against host PostgreSQL 18; the V4 test install and its H2 database were removed 2026-09-28 and no data is migrated	Complete the operator key and certificate ceremony (\$18.3)
ST-10	Ledger ingestion gateway — <code>ao-ledger-ingest</code>	Planned	mTLS validation, authorization, audit, and idempotency specified; not deployed	Deploy behind the adapter boundary once the ceremony is complete
ST-11	Sales and orders — <code>ao-sales</code> database	Implemented	Sales DB deployed; order, receipt, and fulfillment records supported	Confirm the reporting projection
ST-12	Payment adapters — <code>ao-ingress-payment</code>	Planned	Sales DB deployed; payment provider, verifier, and API pending; blocked on the provider decision	Select the provider and implement the verifier and API, including the website email > PDF > Corda intake path
ST-13	Mastodon local stack	Complete (live, federated)	All 5 containers running; web 127.0.0.1:3000 and streaming 127.0.0.1:4000 verified; /api/v1/instance reports mastodon.300x3.com v4.3.7	Fold the <code>always-on-sales</code> (UID 993) placement back to the operator account (\$20.0)
ST-14	Mastodon federation edge — Cloudflare Tunnel	Partial	Dedicated tunnel <code>always-on-mastodon-federation</code> active; HTTP/2 connector up; actor and WebFinger 200; canonical accounts <code>admin@</code> and <code>bot@</code> ; local-to-remote follows confirmed	Complete the signed round-trip and reverse-follow evidence
ST-15	OpenClaw and LM Studio support chat	In progress	Local stack in progress; OAuth/client issues recorded	Complete OpenClaw and local LLM validation (Tokodon removed per ES.1)
ST-16	Konqueror — dedicated automation browser	Implemented	Designated as the automation browser in ES.1; Tokodon removed completely	Retain as the only browser role for automation
ST-17	Archive and IPFS egress — <code>ao-egress-archive</code>	Partially implemented	Local restic backup and restore validation complete; pCloud replication and IPFS transfer not yet exercised	Provision pCloud/archive credentials and test replication
ST-18	Backup and restore	Implemented	Encrypted restic snapshot 548d9910; file hash validated; database 14/14 tables restored; recurring schedule automated 2026-08-31 (restic nightly 03:30, weekly integrity verify Sun 04:30, nightly domain DB dumps 03:00)	Schedule recurring restore tests

ID	Component	Status	Current state	Next action
ST-19	Monitoring — Prometheus, node_exporter, Grafana	Implemented (data collection)	Prometheus + node_exporter + Grafana run as scottw Quadlet units on ao-admin; Grafana state is PostgreSQL-backed (/api/health reports database: ok); Prometheus is its only datasource; all targets scrape up	Build the dashboards and alert rules; Prometheus acts alone and independently per ES.1
ST-20	Metabase ad-hoc reporting	Implemented (login surface); application database outstanding	Runs on the host and serves its login page in the browser; supersedes the earlier "not serving" finding, which described a different, undeployed container. Metabase performs ad-hoc read-only reporting across the other PostgreSQL and MySQL databases, and over local desktop-application SQLite files, so it needs a dedicated PostgreSQL application database of its own for its schema, saved questions, dashboards, and subscriptions	Provision the Metabase application database and the per-source read-only reporting roles; then confirm state survives restart and the first protected read-only query succeeds with no source writes (WORK 000601)
ST-21	QGroundControl mission planning	Planned	Desktop primary planning with a KaliOS RPi5 fallback; headless simulation and the ROS-Gazebo bridge verified	Add scenario and QGroundControl validation as needed
ST-22	Field gateway and link-quality display	In progress	Heltec V3 connection and a stable serial path verified 2026-08-31; gateway service deployment pending	Deploy the ao-field gateway service (WORK 000700 evidence)
ST-23	Reporting and database administration identities	Planned	PostgreSQL is loopback-only. Metabase needs one read-only role per reporting source; Grafana reads approved existing datasources; both keep their own application databases separate from every source database	Define the Metabase application database, the per-source least-privilege read-only roles, and the Grafana/Metabase administration roles and views
ST-24	KDE Wallet secret delivery to Quadlet services	Implemented	Services consuming Wallet secrets start after Plasma login; the ~60s wait is the bounded startup allowance	Confirm the org.kde.kwallet6 bus and method names on the running host (§14.1.1)
ST-25	GPU scheduling and admission	Planned	Driver/CDI verified; CPU baseline and GPU smoke completed	Test scheduling and admission against the ES.1 priority policy
ST-26	Ledger/Corda operator console	Blocked	Narrow operator-management path specified; no public access	Open after the key/certificate ceremony (§18.3)
ST-27	Payment-provider dashboard	Blocked	Provider-hosted, provider-authenticated workflow	Open on payment-provider selection (ST-12)
ST-28	Home automation — Domoticz on RPi	Planned	Specified in ES.1 as the usual Domoticz feature set including cameras and weather	Provision and enumerate the device classes
ST-29	GUI-less controlled data services (ao-data)	Implemented as intentional design	Host services remain loopback-only; administration uses dedicated host or ao-admin identities	None; retain as designed

The GUI and workflow boundary matrix in §19 and the validation evidence table in §20 both reference these IDs. Where those tables record *evidence*, that evidence is preserved there; only the status itself is held here.

ES.3.2 Current Work

ID	Area	Next action	Acceptance criteria	Priority
000800	Vehicle simulation	Stand up the 3D world, build the boning/alignment and datum structure, add reinforcement learning objects, and expose the environment through an HTML portal to operation (§10.1, ES.1)	The world can be created, boned, and driven from the HTML portal in a browser; an RL object is trainable in the world	High
000801	Fabrication and facility simulation	Stand up the 3D world, build the boning/alignment and datum structure, add reinforcement learning objects, and expose the environment through an HTML portal to operation (§10.2, ES.1)	The world can be created, boned, and driven from the HTML portal in a browser; an RL object is trainable in the world	High
000600	Mastodon	Confirm authoritative service account, loopback origins, Cloudflare route, egress isolation, and OpenClaw publisher	Health/WebFinger/actor/origin/egress/API publisher/audit-PDF path pass	Normal
000601	Metabase	Confirm persistence and first protected read-only reporting path	State survives restart; read-only role/view query succeeds; no source writes	Low
000700	MeshChatX/RNode	Observe and characterize the observed minor RF interference	Evidence recorded either way: if the interference is measurable, capture RSSI/SNR/noise/loss/retry/airtime/cross-band data; if it is benign ambient noise, record that finding. No corrective action is required unless measurement shows a real fault. Closure = a recorded characterization, not a fix	Low
—	Prometheus	Deploy TSDB/exporters/rules/alerts for security	Alert tests pass with Prometheus acting alone and independently of Grafana and Metabase	Normal
—	Corda	Complete key/certificate and synthetic ledger test	Confirmed state in dedicated Corda DB/projection	Normal
—	WebODM folders	Create/validate tree, ownership, sentinel, checks	WebODM starts only with required validated storage	Normal
—	Payment adapters	Build provider/reconciliation evidence path, including the website intake path	Valid evidence from PayPal, Zelle, or Coinbase — and from website email > PDF > Corda processing — produces normalized state and an eligible ledger manifest	Normal
—	Wallet/Quadlet	Test login/restart/re-login delivery	Services receive named secrets only after KDE login	Normal
—	GPU	Test scheduling/admission	LM Studio/SketchUp/Gazebo/WebODM match policy	Normal

ES.4 Detailed System Record — Start of the README

Architecture, Installation, Configuration, Operations, and Status

This is the single authoritative document for the ALWAYS ON project. It contains the intended architecture, implemented configuration, operational requirements, validation evidence, approved deviations, known issues, and work queue. The remaining tables, commands, evidence, paths, full interface inventory, operational requirements, payment/ledger forms, Mastodon details, installation procedures, backup rules, and service records follow, numbered 1–21. Where an older statement conflicts with ES.1, ES.1 governs. When the current implementation differs from an architecture requirement, the difference must be recorded in **Approved Deviations and Open Decisions** with a rationale, compensating controls, owner, and resolution condition.

Last consolidated review: 2026-09-30

Operator warning. Everything here is difficult until it is easy. These tools are sharp and dangerous, and if you disrespect them they will can/will harm you — the same as any trip, fall, or car ride. Perhaps not as bad, perhaps worse. In my opinion, ignoring them is more dangerous than understanding them.

1. System Purpose

ALWAYS ON is a compartmentalized, on-premises platform supporting an automated approximately 160-square-foot modular live/fabricate facility and an accompanying modular micro-aircraft carrier. Both of which grow to generally any size / quantity.

The platform supports:

- Drone telemetry and field communications.
- Photogrammetry and mapping.
- Vehicle simulation.
- Home automation.
- Fabrication, facility, inventory, kitchen, and logistics simulation.
- Static HTML & interactive iframe content from other servers.
- Hosted payment checkout and receipt generation.
- Corda-backed provenance, receipts, entitlements, and approved state records.
- Encrypted pCloud archival replication and controlled IPFS artifact distribution.

The current workstation is a development, integration, and validation host. It uses Kubuntu 26.04 LTS software, an AMD CPU, and an EVGA NVIDIA GTX 1080. Future compute-intensive production workloads may move to an immersion-cooled server rack and a Raspberry Pi edge-computing cluster.

Kubuntu was selected for the current workstation role for four primary reasons:

- **Commercially stable with long-term service.** It is built on Ubuntu LTS, whose standard support is scheduled through April 2031, so the workstation has a predictable, vendor-backed maintenance horizon rather than a rolling-release one.
- **First-class ROS 2 support for robotics work.** It carries the ROS 2 and Gazebo toolchain this project depends on for vehicle and fabrication simulation, plus QGroundControl for mission planning and GPU diagnostics for the GTX 1080.
- **The KDE Plasma desktop environment.** Plasma is the shell this project is designed around — it provides the login-gated KDE Wallet secret flow in §14.1.1 and Konqueror as the dedicated automation browser.
- **The huge KDE suite of software and related personal-computing hardware.** This matters most for laptops: Linux on comparable hardware runs roughly four times better than similarly priced non-Linux machines, and the KDE suite covers the desktop and portable use that the system is built for.

These are the primary reasons it was selected.

Podman is the only supported container runtime. Containers are managed through systemd Quadlet definitions rather than Kubernetes, shell wrappers, Docker Compose, or any other orchestration mechanism or a Docker daemon.

2. Platform Baseline

2.1 Intended Platform Standard

Area	Architecture requirement
Host OS	Kubuntu 26.04 LTS workstation — selected for commercial stability and long-term service on an Ubuntu LTS base (support through April 2031), first-class ROS 2 support for the robotics/simulation work, the KDE Plasma desktop environment, and the huge KDE software suite together with compatible personal-computing hardware (Linux laptops run ~4× better than similarly priced non-Linux machines). These are the primary reasons it was selected (§1)
Current CPU/GPU	AMD CPU and EVGA NVIDIA GTX 1080
Future compute	Immersion-cooled server rack and Raspberry Pi edge cluster
Container engine	Podman only
Container lifecycle	systemd and Podman Quadlet
Public website	Static HTML in pCloud Public Folder + Cloudflare domain registration
Payments	Provider-hosted checkout and verified payment events; no local card handling; local stablecoin processing
Sales and support	Sales API, PostgreSQL, Mastodon integration, OpenClaw, and LM Studio
Drone compute	Raspberry Pi 5 with WAVESHARE SX1262-class LoRa top-hat
Drone autopilot	3DR N1 connected to Raspberry Pi 5 by MAVLink
Desktop radio	Heltec WiFi LoRa 32 V3 through stable USB serial path
Field protocol	RNS/Reticulum and MeshChatX over raw LoRa unless a true LoRaWAN deployment is selected
Mapping	WebODM and supporting services under Podman
Mapping storage	/media/scottw/500GBPHOTOGRAM/
Vehicle simulation	ROS 2 Lyrical, Gazebo Sim 10.5.0, ArduPilot SITL, MAVLink, QGroundControl
Fabrication simulation	ROS 2 Lyrical, Gazebo Sim 10.5.0, robot cells, additive manufacturing, storage, kitchen, and logistics models
Ledger	Corda core behind a dedicated ledger-ingestion gateway
Archive	Local source data, signed manifests, encrypted pCloud replication, private or encrypted IPFS workflow
Monitoring	Prometheus-compatible metrics, alerts, health checks, and protected administration access
Backup	PostgreSQL/Corda-aware backup, restic or equivalent encrypted backup, and scheduled restore testing

2.2 Current Host Facts

Area	Verified current value
Kernel	7.0.0-34-generic
Podman	5.7.0
Podman networks	Ten ao-* networks present; internal workload-domain isolation verified
GPU	EVGA NVIDIA GTX 1080
NVIDIA driver	580.178.04
NVIDIA integration	CDI devices registered, including nvidia.com/gpu=0
Simulation stack	ROS 2 Lyrical at /opt/ros/Lyrical; Gazebo Sim 10.5.0
Host PostgreSQL	PostgreSQL 18.6, loopback-only
Host Redis	Redis 8.0.5, loopback-only
Mapping drive	ext4 /dev/sdb1; UUID verified; approximately 433.9 GB free of 457 GB
Mapping mount	/media/scottw/500GBPHOTOGRAM/
Desktop OS	Ubuntu 26.04.1 LTS (resolute) userland with the Kubuntu desktop
Current runtime model	Mixed rootless and system/rootful Podman evidence; see approved deviation section

Reticulum and MeshChatX host facts are recorded in §9, not here.

3. High-Level Architecture

3.1 Isolation Detail View

This is a **detail view of the ingress/egress portion of the master topology in ES.2** and must not contradict it. For the whole project, read ES.2.

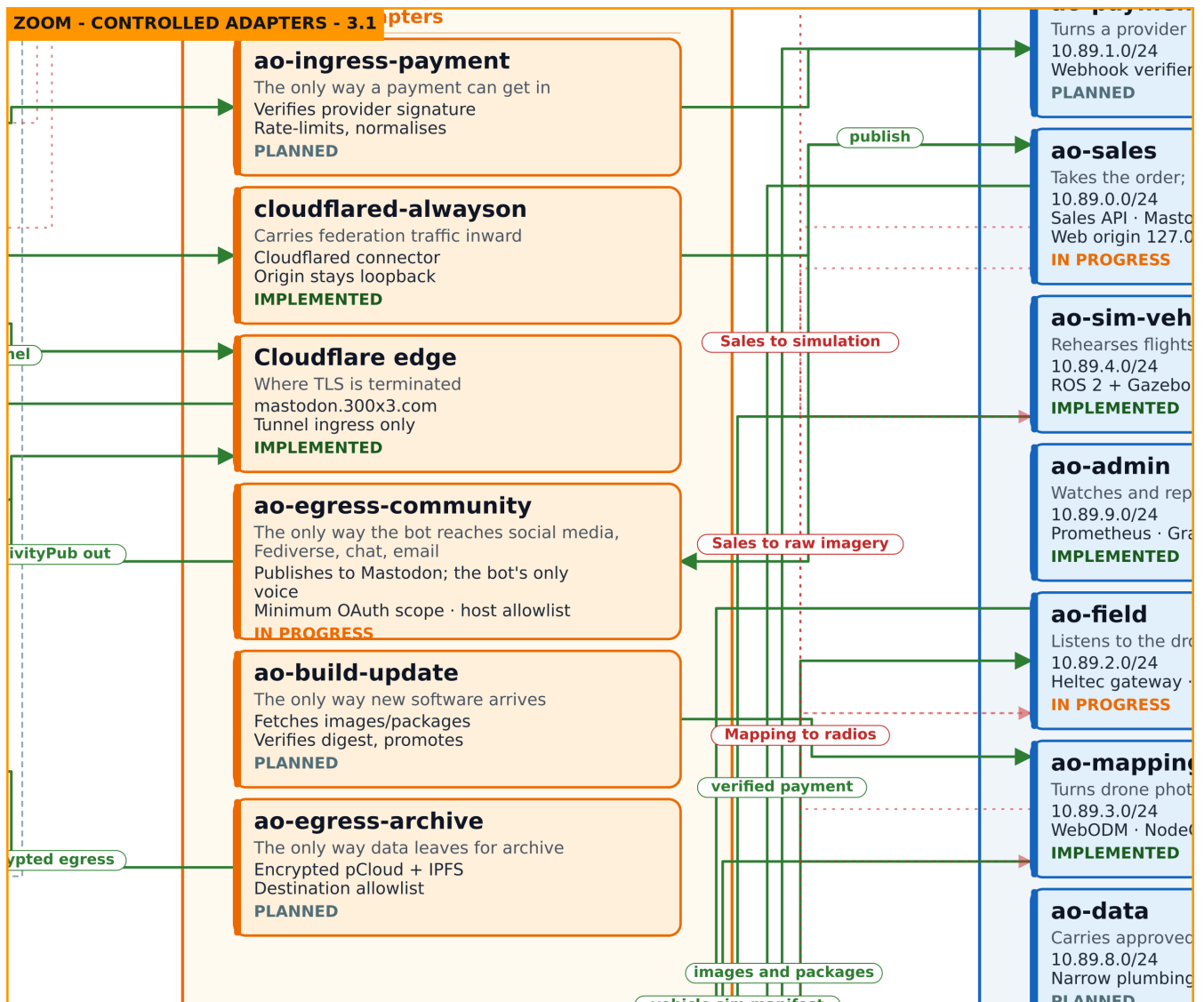


Figure 3.1 — A zoom of the controlled adapter column of the same master topology graphic used in ES.2, enlarged so the labels are readable. It is a zoom of that one graphic, not a separate diagram: the adapter column is the only place a controlled ingress or egress process may exist, and every route into or out of the host passes through it.

The public storefront has no direct route to the Kubuntu host's field, mapping, simulation, database, AI, Podman, or Corda-core services.

3.2 Implementation Status and Change Summary — Recorded Elsewhere

Both of the tables that used to live here have been removed, because keeping a second copy of either one is how the two copies drift apart:

- **Implementation status** is tracked once, as the single list in **ES.3**. That list is the only place a component is given a status. The status columns in §19 (GUI and workflow boundary matrix) and §20 (validation evidence) no longer restate a status; they reference the ES.3 ID for that component, so there is one status per component and one place to change it.

- **The change summary** is recorded once, in **ES.1.1 Change Summary**. Appendix A is the revision index only and carries no change summary.

Neither table is repeated anywhere else in this document.

3.3 DATABASES AND DATA STORES

PostgreSQL 18 is the system-wide relational database platform. The target is one host-managed PostgreSQL installation with separate logical databases and separate application roles. During the current migration, some applications still run container-scoped PostgreSQL instances; those are current-state implementations, not the permanent architecture. The database name and role boundary remain unchanged.

Target logical databases:

salesdb	# Authoritative sales/payment records
mastodon	# Mastodon application state
webodm	# WebODM/PostGIS mapping data
cordadb	# Corda 5 with a dedicated Corda PostgreSQL database
postgres	# Administrative/maintenance database

Grafana and Metabase each keep their own application database. Grafana is for stable, long-lived dashboards and metrics. Metabase is for ad-hoc reporting by users.

The two tools differ in *kind*, not merely in label:

- **Grafana** holds **stable dashboards** — curated, versioned, reviewable views that are built once and kept. It is the tool for a number that must stay on a wall or in a briefing. It reads its approved datasources (Prometheus, and approved existing PostgreSQL databases) **read-only** and renders them as dashboards and metrics. Grafana does not write into any database it reads.
- **Metabase** holds **ad-hoc reports** produced by users exploring data — a question asked in the browser, saved, filtered, and exported. A saved Metabase question is the unit of work, not a dashboard.
- **The progression is intentional.** A recurring ad-hoc Metabase report that proves its worth is expected to be **promoted into a Grafana dashboard**, where it becomes stable and curated. Ad-hoc first; promote to a stable dashboard once it is worth keeping.

Metabase also develops ad-hoc reports over **SQLite** databases belonging to desktop applications (MeshChatX, QGroundControl, Akonadi/KDE PIM, and browser profile stores), read-only and local to the host. DBEaver remains the hands-on desktop surface for the same SQLite files; Metabase is the user-facing reporting surface over them.

This is a correction to the earlier statement in this document that neither tool needs a dedicated application database. Both do.

- **Metabase** performs ad-hoc read-only reporting across the other PostgreSQL and MySQL databases. To do that it must persist its own state — the Metabase application schema, saved questions, dashboards, filters, subscriptions, and the report cache — so it is given a **dedicated PostgreSQL application database** of its own. That database holds Metabase's configuration and saved work only. It is **not** a system of record for any business data, it holds no sales, payment, or ledger record, and it must never be written to by the reporting sources. Metabase reaches the business databases through **read-only** database users, one per source, so that a reporting query cannot modify a source even if a query is written badly.
- **Grafana** likewise keeps its own PostgreSQL-backed application database for users, dashboards, and datasource configuration, as is already recorded in §20.
- Neither tool is a database of record. The authoritative operational data stays in the business databases and in Corda, and the tools read it. Their own databases hold configuration and saved work, not business data owned by them.

Redis is a low-latency speed and coordination layer, not the authoritative system of record. It is used for caching, queues, locks, task brokering, and transient operational coordination. Important business, payment, user, and application metadata remain in PostgreSQL.

Prometheus is for security only. It operates independently of the other systems to ensure security and to address any problems, and keeps its own time-series store. It is not replaced by PostgreSQL, Grafana, or Metabase, and it does not depend on either of them.

SQLite and H2 are not approved application databases for Grafana or Metabase; both use PostgreSQL for their application state. They may exist only where an individual application requires embedded local storage — the browser, Akonadi, Podman, MeshChatX, QGroundControl — or in retained migration backups.

Database/ledger authority

- **AUTHORITATIVE RELATIONAL:** PostgreSQL owns detailed operational data, searchable business records, and reporting projections.
- **AUTHORITATIVE LEDGER:** Corda owns the complete ledger of debits and credits, together with final sale/contract state, receipt association, entitlement, and approved ledger state transitions. Where PostgreSQL holds the detailed operational and searchable business record, Corda holds the authoritative debit and credit position itself.

3.3.1 Program-to-Database Map (single consolidated table)

This is the one table of programs and the databases they use. It is the starting point for discussing reporting and data integration; it is not a list of every installed package or desktop settings module.

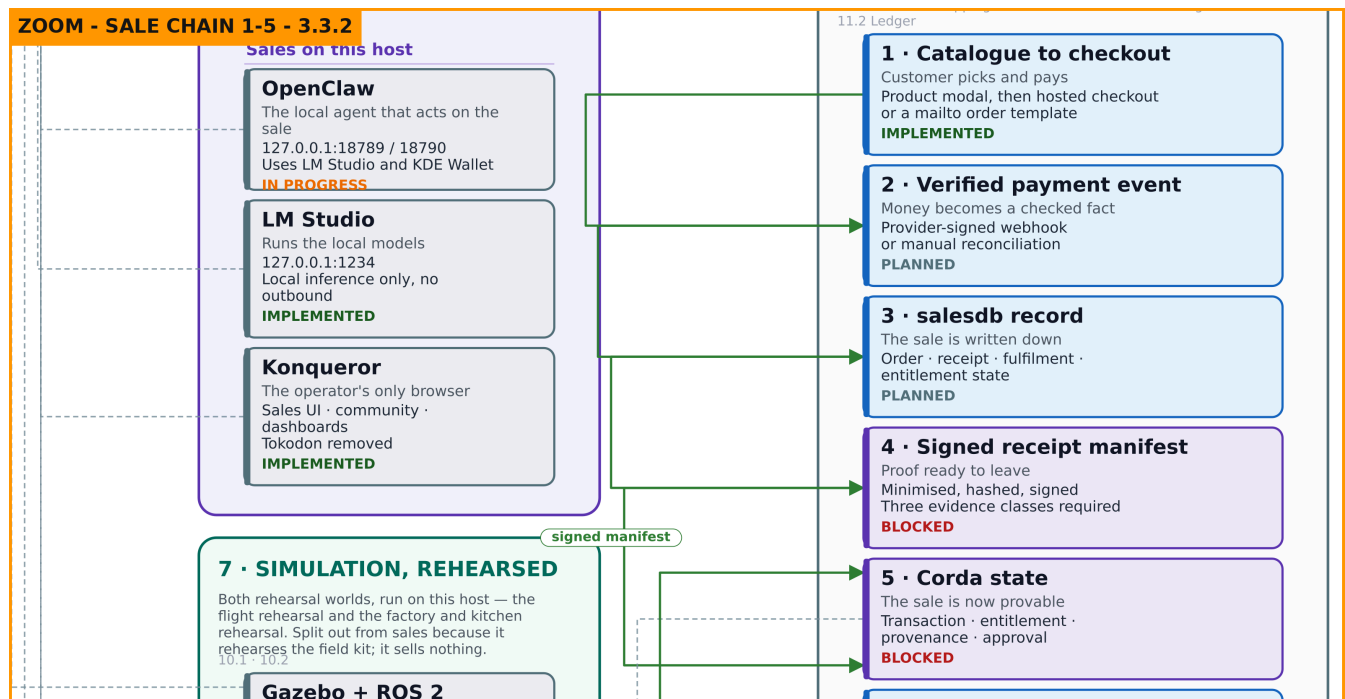
Database software	Software/program	Database name or store	Current role and reporting value
PostgreSQL 18	Host PostgreSQL service	Host cluster; postgres	Shared relational platform and administrative/maintenance cluster
PostgreSQL 18	Sales database service	salesdb in sales-db	Authoritative source for customers, orders, products, payments, receipts, entitlements, and audit history
PostgreSQL 18	Mastodon web/background workers	mastodon in mastodon-db	Accounts, posts, media metadata, federation state, and background-job application data
PostgreSQL/PostGIS	WebODM web/worker	webodm or webodm_dev in db	Mapping projects, processing state, users, and geospatial data
PostgreSQL/PostGIS	NodeODM	WebODM PostgreSQL plus filesystem processing data	Processing-node state and coordination; large image/output artifacts remain filesystem data
PostgreSQL 18	Corda 5 node	cordadb (dedicated Corda PostgreSQL database in the host cluster, per ES.1)	Receipt, entitlement, and provenance state. Built on Corda 5 against cordadb; the previous V4 test installation and its H2 database were removed 2026-09-28 and no data needed to be migrated.
Redis 8	Host Redis service	Host Redis database 0	General low-latency cache/coordination layer; no current application data confirmed
Redis 8	Mastodon cache/queue service	mastodon-redis database 0	Cache, queues, and background-job coordination; not authoritative business data
Redis 8	WebODM broker	broker database 0	Celery/task broker and worker coordination; not authoritative mapping data
PostgreSQL 18	Grafana	Grafana application database (dedicated)	Grafana users, dashboards, and datasource configuration. Confirmed PostgreSQL-backed against the host cluster over the /var/run/postgresql socket; /api/health reports database: ok. Its own application state, not business data
PostgreSQL 18	Metabase	Metabase application database (dedicated)	Metabase application schema, saved questions, dashboards, filters, and subscriptions. Not a system of record and never written to by a reporting source
PostgreSQL / MySQL (read-only)	Metabase reporting sources	Per-source read-only roles	Ad-hoc read-only reporting connections to the business databases. One read-only role per source, with no write, DDL, or owner privilege, so a report cannot modify a source
Prometheus TSDB	Prometheus	/prometheus persistent volume	Security evidence only. Time-series metrics, service health, resource usage, and security evidence. Acts alone and independently of Grafana and Metabase.
SQLite	MeshChatX	MeshChatX SQLite store	MeshChatX messages, rooms, and local Reticulum/MeshChatX application state
SQLite	QGroundControl	QGroundControl SQLite store	QGroundControl plans, waypoints, settings, and vehicle/flight-plan state
SQLite	Akonadi/KDE PIM applications	Akonadi SQLite data	Contacts, calendars, mail indexes, and local personal-information data
SQLite	Firefox, Brave, Chrome, and Edge	Browser profile SQLite stores	Browser history, site storage, caches, certificates, and profile data
SQLite	Podman	Rootless container metadata store	Container, image, network, and volume metadata; not application data
Filesystem/local metadata — social	LM Studio and OpenClaw	Application files, model settings, and local state	Sales/social AI application state that is not automatically part of SQL reporting
Filesystem/local metadata — drone/field	ArduPilot, MeshChatX/Reticulum field stores, and radio gateway logs	Application files, telemetry spools, and local state	Field operational and engineering data that is not automatically part of SQL reporting
Filesystem/local metadata — sim	Gazebo, ROS 2, ArduPilot SITL, and simulation tools	Project files, worlds, models, and result artifacts	Simulation operational and engineering data that is not automatically part of SQL reporting

This table describes databases, not work. Where a database is not yet built, initialized, or provisioned, that is outstanding work and it is **not** stated here — it is tracked in **§19** as a numbered implementation item, with its status in **ES.3**. This table records only what each database is and which program uses it.

3.3.2 Data Flow into Reporting and Ledger Records

Corda is not a replacement for the source PostgreSQL databases. The intended ledger flow is:

The sale chain on the right of the following extract — customer picks to checkout, verified payment event, salesdb record, signed receipt manifest, and Corda state — is the same five-step chain drawn in ES.2. Payment intake is shown at the left, through `ao-ingress-payment` and into `ao-sales`; the reporting projections (Metabase, Grafana) and the Corda entry are shown in their columns.



Zoom of the ES.2 master topology, cropped to the five-step sale chain and enlarged so every label is readable.

Where the blockchain entry is recorded. The entry itself is created by Corda 5 in `cordadb` (§3.3, §18.2). The approved projection of that state is recorded in the corresponding PostgreSQL database, keyed by the same transaction/receipt reference, and the PDF receipts and reports that describe it are written to the transaction folder under `/ALWAYSON/data/sales/transactions/<transaction-id>/`.

Use stable correlation fields in the source-to-ledger integration, including a transaction/order reference, serial number where applicable, UTC timestamp, event type, status, and content hash. Do not copy payment credentials, private keys, or unrestricted customer datasets into Corda. Corda remains authoritative for approved ledger/provenance state, while PostgreSQL remains authoritative for domain-operational source data.

Before building sales reporting, initialize and verify the `salesdb` schema and confirm that the reporting identity can read it and produce the standard PDF reports and receipts where appropriate. Before enabling blockchain-related flows, complete the Corda key/certificate ceremony and implement the ledger-ingest manifest, correlation-ID, signature, idempotency, and audit requirements.

4. Security, Isolation, and Data Policy

4.1 Non-Negotiable Rules

1. Inspect before changing.
2. Preserve existing data.
3. Never format, repartition, delete, prune, or overwrite without explicit operator approval.
4. Never install Docker daemon, Docker Compose, Watchtower, or Kubernetes.
5. Use Podman and Quadlet only.
6. Never expose a public port without explicit operator approval.
7. Never place secrets in scripts, logs, HTML, Git, pCloud Public Folder, IPFS, Corda payloads, shell history, or documentation examples. Only use KDE Wallet for passwords, tokens, keys, and other secret material.
8. Never use `--privileged` as a default.
9. Use pinned image digests for operational services.
10. Verify the photogrammetry drive before deploying or operating WebODM.
11. Record commands, versions, significant output, and failures in the installation or operational journal.
12. Stop and report conflicts involving services, packages, networks, mounts, ports, serial devices, firewall policy, or existing data.
13. Do not broaden network access, database privileges, filesystem access, container privileges, or secret access merely to bypass an error. A documented local integration path with least-privilege credentials is permitted when it is required for PostgreSQL reporting, backup, health checking, or application migration.
14. Require explicit human approval before initiating payments.
15. Require explicit human approval before publishing external communications, changing production credentials, deleting data, or modifying external records.

4.2 Data Classification

Classification	Examples	Handling requirement
Public	Anything in the pCloud Public Folder, or linked in from it: storefront HTML, intentionally published documentation, approved product data	May be placed in pCloud Public Folder
Internal operational	The databases — PostgreSQL and SQL — plus non-sensitive configuration, health data, non-sensitive manifests, unit status	Restricted local access; do not publish by default
Sensitive	Corda blockchain secured data and the accounting ledger, plus customer contact data, payment references, precise telemetry, sensitive imagery, proprietary technical designs	Domain-restricted storage; encrypted backup; no public IPFS
Secret	Passwords, held in KDE Wallet	KDE Wallet only; delivered to services after Plasma login by design (§14.1.1)

4.3 Prohibited Paths

The full flow is drawn in the master topology graphic in ES.2. The five numbered steps of the sale chain — catalogue to checkout, verified payment event, `salesdb` record, signed receipt manifest, and Corda state — are steps 1 to 5 in the highlighted column below, which is that same graphic with the chain highlighted:

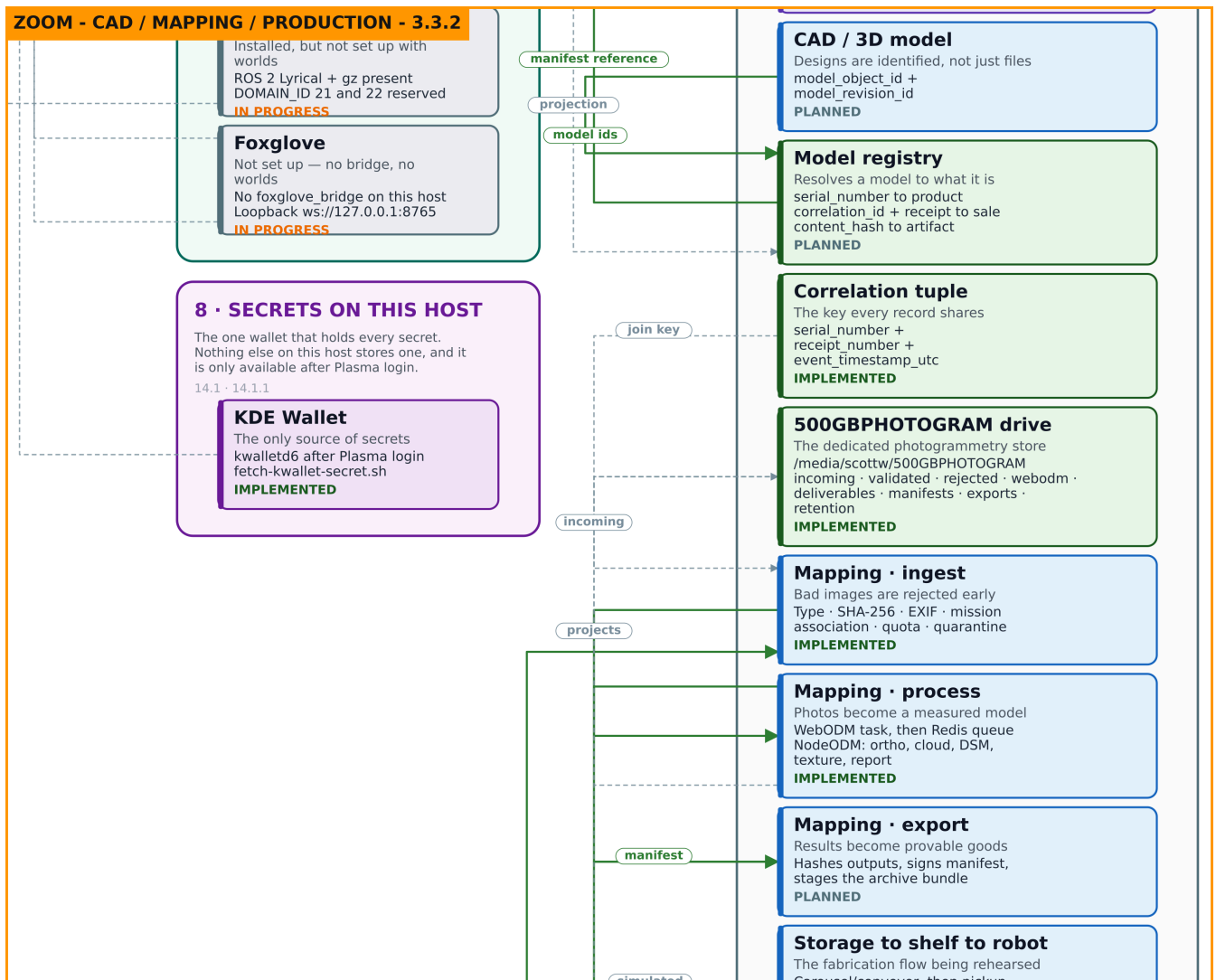


Figure 3.3.2 — A zoom of the catalogue-to-ledger chain from the ES.2 master topology, enlarged so the labels are readable. It is a zoom of that one graphic, not a second diagram.

Two paths feed that chain, and both are in the same graphic. The **PDF intake path** runs from KIT REQUEST PDF intake into ao-sales ("PDF requests in"), which is the public storefront intake. The **website path** runs from the customer email inbox through the Cloudflare edge and tunnel into ao-sales as a loopback origin. From there ao-sales emits a **signed receipt manifest** into ao-ledger-ingest, which performs mTLS, authorization, idempotency, and audit before passing an **approved state transition** to ao-ledger-core. Ledger state reaches the reporting tools by two separate approved routes, both drawn in the graphic: ao-ledger-core supplies status over a narrow API, and salesdb supplies **read-only views** to Metabase.

4.4 Approved Internal Paths

PDF INTAKE

Website KIT REQUEST / order-request email → Public PDF intake form
→ standardized PDF request → kit-request-intake/ inbox
→ extracted text + manifests → operator review → transaction bundle

Sales receipt PDF → Ledger-ingestion gateway
Verified payment event → Sales API and/or ledger-ingestion gateway
Field telemetry manifest → Ledger-ingestion gateway
Mapping deliverable manifest → Ledger-ingestion gateway
Vehicle simulation manifest → Ledger-ingestion gateway
Fabrication simulation manifest → Ledger-ingestion gateway

PDF OUTPUT

Signed receipt / contract PDF → Transaction folder + operator archive
Standard sales report PDF → ao-admin reporting output
Standard accounting/ledger report PDF → ao-admin reporting output
Mapping deliverable report PDF → Deliverables folder + customer transfer
Field telemetry report PDF → Field reporting output
Simulation result report PDF → Simulation results directory

Ledger receipt or entitlement status → Authorized service through narrow API
Signed mission release → Field mission-release service
Validated image set → WebODM intake service

All cross-domain requests require:

- Mutual TLS.
- A dedicated service certificate or identity.
- Signed payload where durable provenance is required.
- Schema validation.
- Timestamp and nonce or equivalent replay defense.
- Durable idempotency key handling.
- Audit record.
- Explicit authorization policy.

Mutual TLS authenticates transport peers. Detached manifest signatures permit independent verification after storage, export, or audit. These are separate controls and should be used together for provenance-bearing artifacts.

5. Network Domains and Controlled External Access

5.1 Combined Domain, Adapter, Component, and GUI Matrix

This is the single table for network domains, controlled ingress and egress adapters, component boundaries, and GUI/workflow attachment. Sections 5.2, 6.1, and 6.A.1 formerly held four separate tables covering the same ground; they are combined here so that a component's domain, its inputs, its outputs, its data, its exposure, and its status are read from one row rather than reconciled across four. The gray bars separate the four groups, which are **A** workload domains, **B** controlled ingress and egress adapters, **C** the component boundary matrix, and **D** the GUI and workflow attachment map.

Attachment rule (applies to every row in group D). Each GUI or workflow is attached to exactly **one** owning `ao-*` network and is denied attachment to all the others. The denied set is always the complement of the owning network, so it is not repeated per row. Read-only access to a second network is permitted **only** where the approved access path says so explicitly, and never as a broad membership.

An **associated domain** identifies the operator workflow a tool serves. It does not grant broad Podman-network membership, database access, host access, shared storage, shared credentials, or cross-domain control. A host desktop application, host browser, or external provider dashboard has no Podman network attachment unless it is itself implemented as a container attached to that network.

`ao-admin` is the protected administration, monitoring, and reporting plane. It may host Grafana for stable dashboards and metrics, Metabase for ad-hoc reporting, and narrowly authorized administration tools. It must not become a shared universal network. `ao-data` remains narrow controlled data plumbing, not a default GUI, shared-database, or reporting network.

All workload-domain networks are `Internal=true`. CIDRs are recorded in:

```
/ALWAYSON/config/platform/network-cidrs.yaml
```

No workload service may receive unrestricted Internet access simply by joining its application-domain network.

External connectivity is allowed only through the narrowly scoped, independently reviewed adapters in group B. These adapters are architecture-controlled exceptions, not general-purpose Internet access. No sales, mapping, field, simulation, database, AI, or ledger-core container may attach directly to an Internet-capable network. An external adapter must use separate credentials, destination allowlists, validated DNS/TLS, firewall policy, minimal permissions, and connection logging.

ao-egress-community is removed. The Mastodon/community publication and federation work is already located inside `ao-sales`, which keeps the Mastodon web and background-worker containers on the internal sales network with their own controlled egress bridge. A separate community adapter network is therefore not required and is not part of the design. This is recorded as group C row "Community publication".

Item	Owning domain / network	Purpose, inputs accepted, or approved access path	Outputs allowed / permitted output	Persistent data	External connectivity / public exposure	Status — see ES.3
A · WORKLOAD DOMAINS — every row is an Internal=true Podman network; CIDRs in /ALWAYS0N/config/platform/network-cidrs.yaml						
ao-sales	ao-sales	Coordinates social media, email, and Mastodon; the AI bot and chat (OpenClaw/LM Studio); and the order-request and receipt workflows, including the public PDF intake forms and PDF output. The only domain that touches customers directly.	Signed order, receipt, and entitlement manifests; standardized PDF intake and PDF output; published federation and chat posts	Sales PostgreSQL	The public PDF intake forms are the exposure point; Mastodon/chat arrive via the Cloudflare Tunnel to loopback origins	ST-11, ST-13, ST-15
ao-payment	ao-payment	Provider webhook verifier and payment adapter	Verified normalized payment state	Minimal event and audit record	No direct public exposure	ST-12
ao-field	ao-field	Heltec gateway, RNS/MeshChatX, telemetry spool, mission-release service	Signed telemetry and mission manifests	Raw packet store and telemetry spool	No direct public exposure; USB serial and radio only	ST-04, ST-22
ao-mapping	ao-mapping	WebODM, NodeODM, Redis, mapping DB, imagery intake/exporter	Signed mapping deliverable manifests	Dedicated photogrammetry volume	No direct operator/VPN access. Input is 100% by drone and automated WebODM processing	ST-03
ao-sim-vehicle	ao-sim-vehicle	ROS 2, Gazebo, ArduPilot SITL, MAVLink, QGroundControl simulation	Signed vehicle-simulation manifests	Vehicle simulation data path	No direct public exposure	ST-07
ao-sim-fabrication	ao-sim-fabrication	ROS 2, Gazebo; coordinates all industrial engineering and production related details and receives specific production data from each machine (MainsailOS, Moonraker, Klipper, 3D printers, CNC); runs the kitchen	Signed fabrication-simulation manifests	Fabrication simulation data path	No direct public exposure	ST-08
ao-ledger-ingest	ao-ledger-ingest	mTLS validation gateway, authorization, audit, idempotency	Corda receipt IDs and status	Audit and idempotency state	No direct public exposure	ST-10
ao-ledger-core	ao-ledger-core	Corda node, Corda database, certificate/keystore material	No direct output	Corda state and PKI	No direct public exposure	ST-09
ao-data	ao-data	Narrow controlled data plumbing where unavoidable	Controlled references only	Host services, loopback-only	No direct public exposure	ST-29
ao-admin	ao-admin	Prometheus security monitoring, Grafana dashboards, Metabase reporting, backup, restore validation, administration	Metabase reports and the Grafana dashboard only	Prometheus TSDB; Grafana application database; Metabase application database	No VPN, no explicit allowlist, no public exposure	ST-19, ST-20
B · CONTROLLED INGRESS AND EGRESS ADAPTERS — architecture-controlled exceptions, not general-purpose Internet access						
ao-ingress-payment	ao-payment	Payment verification for Zelle, PayPal, and Coinbase. Receives the provider webhook or approved relay event, verifies the signature, normalizes it, and emits the verified payment event. Also carries the website path: email > PDF > Corda processing. Planned — not yet deployed	Verified normalized payment event	Minimal event and audit record	Inbound only. Minimal listener, provider-signature verification, rate limits, audit log, normalized event output	ST-12

Item	Owning domain / network	Purpose, inputs accepted, or approved access path	Outputs allowed / permitted output	Persistent data	External connectivity / public exposure	Status — see ES.3
ao-egress-archive	ao-sales (archive duty)	Moving large data and the image/map/telemetry files into IPFS for transfer after sale , plus encrypted pCloud replication. "Data sales": maps and telemetry/IoT products, not application databases. Planned — not yet deployed	Approved encrypted archive bundle; replication result/status	Staging and transfer log	Outbound only. Destination allowlist, TLS validation, encrypted payloads, separate credentials, transfer audit	ST-17
ao-build-update	ao-admin (build/update duty)	Software updates . Image and package acquisition before controlled promotion. Planned — not yet deployed	Verified image and package set	Update audit log	Outbound only. Verified source, digest capture, update audit, no direct workload attachment	ST-01
C · COMPONENT BOUNDARY MATRIX — what each component may accept, emit, store, and reach						
Sales API	ao-sales	Verified payment state and approved support requests	Signed receipt/entitlement manifests	Sales PostgreSQL	None directly	ST-11
Payment verifier	ao-payment	Provider webhook or approved relay event	Verified normalized payment event	Minimal event and audit record	Through ao-ingress-payment only	ST-12
Mapping intake	ao-mapping	Authenticated imagery upload	Validated image-set reference	Intake, validation, quarantine record	None directly	ST-03
WebODM/NodeODM	ao-mapping	Validated mapping task input	Processing output to mapping exporter	Dedicated photogrammetry volume	None directly	ST-03
Field gateway	ao-field	USB serial LoRa frames	Normalized telemetry manifest	Raw packet store and telemetry spool	USB serial and radio only	ST-04, ST-22
Vehicle simulator	ao-sim-vehicle	Approved scenario/model artifact	Signed simulation manifest	Vehicle simulation data path	None directly	ST-07
Fabrication simulator	ao-sim-fabrication	Approved facility/task model	Signed simulation manifest	Fabrication simulation data path	None directly	ST-08
Ledger ingestion	ao-ledger-ingest	Signed mTLS manifests	Receipt/status response	Audit and idempotency state	Only to ledger core	ST-10
Ledger core	ao-ledger-core	Ledger-ingestion gateway requests only	No direct public output	Corda state and PKI	None directly	ST-09
Archive adapter	ao-egress-archive	Approved encrypted archive bundle	Replication result/status	Staging and transfer log	Outbound only	ST-17
Community publication	ao-sales	Approved publication or support request	Remote delivery/status response	Publication audit log	Outbound only, via the sales-domain egress bridge	ST-13, ST-14
D · GUI AND WORKFLOW ATTACHMENT MAP — each row is attached to exactly one owning network and denied all the others						
1 · Mastodon web / Konqueror client	ao-sales	Approved localhost Mastodon web/streaming origin; loopback-only publication when enabled.	—	—	Loopback origin only	ST-13, ST-14
2 · WebODM browser UI	ao-mapping	Approved loopback WebODM listener; VPN/authenticated access only if separately approved.	—	—	Loopback only	ST-03
3 · QGroundControl simulation client	ao-sim-vehicle	Approved local SITL/MAVLink-router endpoint; ROS_DOMAIN_ID=21; GZ_PARTITION=alwayson_vehicle_sim.	—	—	None	ST-21
4 · Gazebo visualization —	ao-sim-vehicle	Approved vehicle ROS/Gazebo visualization path; separate DDS/interface policy remains	—	—	None	ST-07

Item	Owning domain / network	Purpose, inputs accepted, or approved access path	Outputs allowed / permitted output	Persistent data	External connectivity / public exposure	Status — see ES.3
vehicle		required.				
5 · Gazebo visualization — fabrication	ao-sim-fabrication	Approved fabrication ROS/Gazebo visualization path; ROS_DOMAIN_ID=22; GZ_PARTITION=alwayson_fabrication_sim.	—	—	None	ST-08
6 · LM Studio / OpenClaw support chat	ao-sales (host-local LM Studio) / ao-sales (containerized OpenClaw)	Host desktop use; approved loopback inference endpoint or narrow authenticated bridge only.	—	—	None	ST-15
7 · Grafana dashboards and metrics	ao-admin	No VPN, no explicit allowlist, no public exposure. Grafana presents the dashboards and metrics.	—	—	None	ST-19
8 · Metabase reporting GUI	ao-admin	No VPN, no explicit allowlist, no public exposure. Metabase produces the reports by reading the existing PostgreSQL and MySQL databases over per-source read-only roles, and keeps its own PostgreSQL application database for its saved questions and dashboards.	—	—	None	ST-20
9 · Sales, receipt, fulfillment, entitlement, return, and approved support reporting	ao-admin	Metabase is for reports, reads the existing PostgreSQL and MySQL databases over per-source read-only roles, and keeps its own application database. PostgreSQL inspection uses pgAdmin over an explicit purpose-limited loopback or approved tunneled connection. DBeaver is not used for server-side SQL; it is for SQLite reporting on desktop applications.	—	—	None	ST-11, ST-20
10 · Ledger provenance, receipt, entitlement, approval, release, and ingestion reporting	ao-admin	No VPN. Grafana presents the dashboards and metrics from its own application database plus the approved datasources; Metabase produces the reports by ad-hoc read-only queries against the existing databases.	—	—	None	ST-09, ST-10
11 · Backup/restore status display	ao-admin	Shown on the Grafana dashboard; no VPN, no public exposure.	—	—	None	ST-18
12 · Field gateway / link-quality display	ao-field	Approved USB serial/local diagnostic display or protected Grafana dashboard.	—	—	None	ST-04, ST-22
13 · Ledger/Corda console and maintenance	ao-ledger-ingest	Narrow approved operator-management path after key/certificate ceremony; no public access.	—	—	None	ST-26
14 · PostgreSQL reporting, schema inspection, and controlled administration	host loopback	Explicit loopback or approved narrow tunnel/bridge using a dedicated least-privilege database identity.	—	—	None	ST-23
15 · Redis diagnostic client	ao-data (optional)	Explicit loopback or approved narrow diagnostic path using a scoped Redis ACL identity.	—	—	None	ST-29
16 · Payment-provider dashboard	provider-hosted	Provider-authenticated browser workflow.	—	—	Provider-hosted only	ST-27
17 · No GUI — controlled data services	ao-data (narrow only)	Host services remain loopback-only; administration/reporting uses dedicated host or ao-admin identities and paths.	—	—	None	ST-29
18 · No GUI — payment verifier	ao-payment	Provider-hosted checkout and provider dashboard; local verifier has no GUI.	—	—	None	ST-12, ST-27

5.2 Controlled Ingress and Egress Adapters

Combined into the single matrix in §5.1, group B. The three controlled adapters (ao-ingress-payment, ao-egress-archive, ao-build-update) are rows in that table along with their purpose, direction, and mandatory controls.

5.3 Approved Local Data Paths

The following local paths are normal integration paths and do not require a new architecture decision:

- Application containers to their approved PostgreSQL database endpoint.
- Metabase to its own application database on PostgreSQL, and from there to the PostgreSQL and MySQL reporting sources over per-source read-only roles, where Metabase is for reports. The read-only role is what enforces that a report cannot modify a source.
- Grafana to the existing PostgreSQL databases and to Prometheus, where Grafana is for dashboards and metrics.
- Host administration and backup jobs to PostgreSQL through loopback or an explicitly documented local bridge.
- Application workers to their required Redis queue or broker.

These paths must not become public listeners, must use separate credentials, and must not grant unrelated applications access to each other's owner, migration, backup, payment, or ledger credentials. Network isolation remains a defense-in-depth control; PostgreSQL roles and grants are the primary authorization boundary for database access.

6. Component Boundaries, GUI Reporting Tools, and Operator Access

6.1 Component Boundary Matrix

Combined into the single matrix in §5.1, group C. What each component may accept as input, emit as output, persist, and reach externally is recorded there, on the same row as its owning domain and its implementation status.

6.A GUI Reporting Tools and Podman Network Mapping

This subsection is an **architecture requirement**. It defines the required relationship between operator GUIs, reporting tools, dashboards, desktop clients, external provider dashboards, workload domains, and Podman networks.

Section 19 records what remains to be implemented, documented, and validated for this requirement. It does not redefine, weaken, or replace it.

6.A.1 GUI ↔ Podman Network Mapping

Combined into the single matrix in §5.1, group D. All eighteen GUI and workflow rows, each with its one owning network, its approved access path, and its ES.3 status ID, are in that table. The attachment rule that governs them is stated in §5.1 and applies to every group D row.

6.A.2 Reporting Tool Roles

Tool	Primary purpose	Mandatory boundary
Grafana	Stable dashboards and metrics. Long-lived, curated, reviewable views. A recurring ad-hoc Metabase report that proves its worth is promoted here. It reads Prometheus and approved existing PostgreSQL databases read-only as datasources and never writes into them. Covers operational monitoring and visualization: metrics, service health, alerts, queue depth, latency, resource use, storage, GPU state, backup age, restore-test status, certificate expiry, ingest failures, and approved PostgreSQL business/database metrics	Runs in <code>ao-admin</code> ; reads Prometheus for metrics and approved existing PostgreSQL datasources for business/database dashboards. It keeps its own PostgreSQL-backed application database for users, dashboards, and datasource configuration. It does not write into the databases it reads, and never becomes a shell, container-management, or control path.
Metabase	Ad-hoc reporting by users. A recurring report that proves its worth is promoted into a stable Grafana dashboard. Also develops ad-hoc reports over desktop-application SQLite files, read-only. FOSS relational reporting: sales, orders, receipts, fulfillment, entitlements, returns, approved support summaries, ledger/provenance projections, saved questions, filters, exports, and standard PDF reports/receipts	It runs in <code>ao-admin</code> ; reads the existing source databases over per-source read-only roles — one read-only role per PostgreSQL and MySQL source, with no write, DDL, or owner privilege. It keeps a dedicated PostgreSQL application database of its own for the Metabase schema, saved questions, dashboards, filters, and subscriptions, because it performs ad-hoc read-only reporting and must persist that work. It does not write into the databases it reports on, and its application database is not a system of record. It never receives superuser, database-owner, migration, backup, payment-provider, or Corda-key credentials.
Corda management/API/CLI	Corda lifecycle, configuration, certificate-aware administration, and controlled maintenance	Uses a documented narrow management path after the required ceremony; it is not replaced by Metabase or Grafana.
DBEaver	SQLite reporting on desktop applications (the hands-on desktop surface; Metabase is the user-facing reporting surface over the same files). Reads and reports against the desktop applications that keep their data in SQLite — MeshChatX, QGroundControl, Akonadi/KDE PIM, and the browser profile stores. It is not a PostgreSQL administration tool and not a routine accounting or business-reporting surface	Opens the local SQLite file for the desktop application under review. Read-only by default; it is not the surface for server-side SQL databases, which are handled by Metabase for reporting, Grafana for metrics, and pgAdmin for PostgreSQL administration
Payment-provider dashboard	Provider-authoritative charges, refunds, disputes, payouts, exports, and reconciliation	External provider service; no Podman network attachment and no replacement of local verified-event controls.

Metabase may report on approved Corda-derived business and provenance data only through a deliberate read-only reporting projection, approved views, supported status interface, or ledger-ingestion audit/status records. Metabase must not become the primary interface to Corda internal persistence tables, administer Corda, receive Corda private keys/keystores, or create a broad route into `ao-ledger-core`.

6.A.3 Conformance Requirements

All current and future GUI, dashboard, reporting, database-administration, and operator-access implementations must comply with this subsection and Sections 4, 5, 14, and 17.

- Every tool must have a named operator purpose, actual runtime placement, approved data/status source, documented access path, and explicit implementation status.
- Every containerized GUI must have documented Podman-network membership, listener policy, service owner, image digest, authentication method, and least-privilege identity.
- Every host desktop GUI and provider dashboard must be recorded as having no Podman network attachment unless it is actually containerized.
- Reporting identities must enforce read-only access to source databases or services. Grafana and Metabase each keep their own application database and read the business databases over per-source read-only roles, writing to none of them; they need no application database of their own.
- `ao-admin` receives approved PostgreSQL reporting, exporter, status, projection, API, relay, tunnel, or push paths. It must not join every workload network.
- `ao-data` is not a shared unrestricted database, general-purpose shell, or authorization bypass. It may carry narrowly approved local data paths.
- No GUI may add a public listener, broad host networking, unrestricted Podman socket access, `--privileged`, shared writable storage, or unrelated-domain secret merely to simplify deployment or troubleshooting.
- Any material deviation requires an approved deviation record under Section 18 before production declaration.

The machine-readable implementation inventory for this requirement is:

```
/ALWAYSON/config/platform/gui-boundary-matrix.yaml
```

7. Public Storefront and Payment Policy

7.1 Storefront Boundary

The public storefront is static HTML hosted in the pCloud Public Folder. HTML project github: <https://github.com/300x3/HTML-300X3> Static HTML output folder for pCloud: `public/html/` (generated by `scripts/build-html.mjs`; dependency-free pages: index, buildings, vehicles, equipment, maps, discussion, documentation, donate).

7.1.1 Frontend Website Details

Source: HTML-300X3 repo (React + TanStack Router; static export in `public/html/`). Navigation is defined in `src/lib/nav.ts` (`NAV` sections with modal previews, pCloud folder links, and SketchUp/Trimble model links). The pCloud Public Folder mirrors the static export layout.

The site is a single-page-per-section storefront. The hierarchy below is the actual navigation tree: each top-level page, its subsections, and what each contains. Indentation is the site hierarchy.

```

www.300x3.com
├── / Home
│   ├── Hero and project introduction
│   ├── Current status highlights
│   ├── Site navigation (links to every section below)
│   └── Follow-up · Support · Chat links, in that order ($7.1.2)
├── /equipment Equipment – catalog modals
│   ├── Adapter ..... soda threads to 0.5" NPT ("TUBER")
│   ├── Boiler ..... water boiler, power production, chemistry set
│   ├── Pneumatic Speargun Ulu .. Damascus ulu + forearm pneumatic speargun
│   ├── Structural Battery ..... "power sandwich" gas/liquid tank + battery case
│   ├── Appliances ..... 12oz micro-appliances ("app cans")
│   ├── Computer ..... 12oz-can Raspberry Pi case + wireless
│   │       └── HDMI / video-glasses kit
│   └── Camping ..... shopping-list discussion, pricing, where-to-buy
├── /buildings Buildings – catalog modals
│   ├── Furniture
│   ├── ADU ..... 80sf and up
│   ├── Mall
│   ├── Tower
│   └── Concrete Island
├── /vehicles Vehicles – catalog modals
│   ├── Drone ..... air / land / sea
│   ├── Boat ..... micro modular aircraft carrier
│   ├── Personal Vehicle
│   ├── Electric Car Wheel
│   └── Balloon
├── /maps Maps
│   └── Photogrammetry deliverables and map products
├── Digital
│   ├── Images
│   ├── Topography and 3D points
│   ├── "Where's My ___?" locator series
│   └── Route-around-your-county
├── Discussion
│   ├── Mastodon forum
│   └── 300X3@POSTEO.NET
├── /documentation Documentation
│   ├── Intro video
│   ├── Working project-plan PDF
│   ├── Server coding
│   ├── 3D models
│   ├── HUD app
│   ├── Simulations
│   ├── AI links
│   ├── Hardware links
│   ├── Software links
│   ├── Fabrication links
│   └── Raw-material links
├── /donate Donate
│   ├── PayPal hosted button (also accepts all major credit cards)
│   ├── Zelle
│   ├── Coinbase / Stablecoin
│   └── Other – customization and coordination, at 300X3@POSTEO.NET

```

Catalog modals under Equipment, Buildings, and Vehicles each carry the sales action described in the sales-link integration plan below.

Order of follow-up, support, and chat links

The order below is deliberate and is the order presented on the page:

1. Follow-up — order status and follow-up links.
2. Support — support request and help links.
3. Chat — AI-assisted support entry point (OpenClaw), which exposes no private infrastructure.

Mastodon/community links follow these three.

Sales-link integration plan

Each catalog modal under Equipment / Buildings / Vehicles gets a sales action that stays inside the static-site boundary (no secrets, no local ports, no internal hosts — see Section 7.1 prohibitions):

1. Modal shows product images, parts list / detailed drawings link, pCloud folder link, and IPFS digital-asset mark where applicable.
2. A purchase button routes to provider-hosted checkout (PayPal hosted button today; Zelle instructions and Coinbase/USDC flow per Section 18.4 as implemented) or to a `mailto:300X3@POSTE0.NET` order-request template carrying product name, options, and quantity.
3. Checkout completion returns a provider-signed event (or manual reconciliation record for Zelle/wire) into the Section 7.3 sales and receipt sequence; Corda records the receipt/entitlement state per Section 11.
4. No payment-card data, webhook secrets, OAuth tokens, or ledger keys ever appear in the static HTML, pCloud folder, or Git history.

```
pCloud Public Folder
├─ index.html
├─ equipment/
│   ├── index.html
│   ├── adapter/
│   ├── boiler/
│   ├── speargun-ulu/
│   ├── structural-battery/
│   ├── appliances/
│   ├── computer/
│   └─ camping/
├─ buildings/
│   ├── index.html
│   ├── furniture/
│   ├── adu/
│   ├── mall/
│   ├── tower/
│   └─ concrete-island/
├─ vehicles/
│   ├── index.html
│   ├── drone/
│   ├── boat/
│   ├── personal-vehicle/
│   ├── electric-car-wheel/
│   └─ balloon/
├─ maps/
├─ digital/
├─ discussion/
├─ documentation/
│   ├── index.html
│   ├── intro-video/
│   ├── project-plan/
│   ├── server-coding/
│   ├── 3d-models/
│   ├── hud-app/
│   ├── simulations/
│   ├── ai/
│   ├── hardware/
│   ├── software/
│   ├── fabrication/
│   └─ raw-materials/
├─ donate/
├─ support/
├─ community/
├─ legal/
│   ├── privacy.html
│   ├── terms.html
│   ├── returns.html
│   └─ shipping.html
└─ assets/
    ├── css/
    ├── js/
    ├── images/
    └─ downloads/
```

The public site may include:

- Product catalog and documentation.
- Hosted payment checkout links.
- Provider-controlled payment buttons.
- Order follow-up links.
- Support links.

- Chat: AI-assisted support entry points that do not expose private infrastructure.
- Mastodon/community links.
- Shipping, return, warranty, privacy, and legal content.

The public site must never include:

- Payment-provider secret keys.
- Corda keys, RPC credentials, or node addresses.
- Mastodon OAuth tokens.
- pCloud archive credentials.
- IPFS private keys or swarm keys.
- Local hostnames, LAN addresses, Podman ports, or private API routes.
- Database connection strings.
- Drone radio configuration, control endpoints, or flight-control access.
- Internal service certificates, identifiers, or diagnostic output.

7.2 Payment and Settlement Policy

ALWAYS ON does not process, transmit, or store payment-card numbers, CVV values, or payment-provider secret material in the storefront, sales database, Corda, Git repository, logs, pCloud Public Folder, or IPFS.

There are three forms of payment processing:

#	Form	Providers	Intended use	Required control
1	Card / PayPal	Hosted card checkout and hosted PayPal checkout	Standard online transactions	Provider-hosted checkout, signature-verified webhook, no local card handling
2	Wire transfer / Zelle	Wire transfer and Zelle	Approved high-value and direct-to-bank transactions	Manual reconciliation, operator approval, auditable reference record
3	Coinbase / stablecoin (USDC)	Coinbase or similar	Crypto/stablecoin settlement	Documented provider terms, accounting treatment, refund process, and explicit operator approval

The default payment model is provider-hosted checkout. The selected provider is responsible for payment-card capture and authorization. The local payment-verifier service accepts only provider-signed webhook events and stores normalized business state.

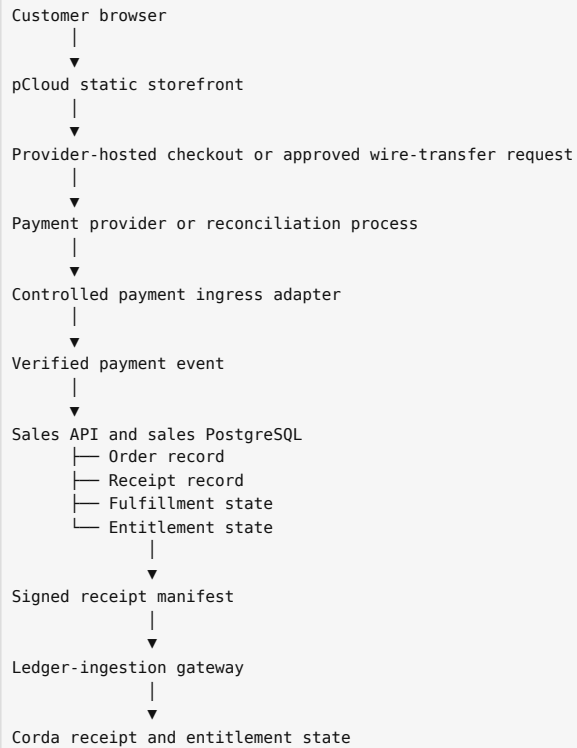
Corda is the central source of truth for financial ledger information, correlated to the PostgreSQL operational database per transaction and per serial number. Corda does not accept payment cards and does not replace the payment provider, banking, tax, consumer-protection, accounting, or refund processing.

Corda does record approved receipt, fulfillment, entitlement, or provenance state after a payment event has been verified or manually reconciled. It must include the related transaction data typical of the financial and payments industry, including the ledger details, and must be queryable by the authorized reporting service — Metabase, which is for reporting, and Grafana, which is for dashboards and metrics.

The three payment forms above — card/PayPal, wire transfer/Zelle, and Coinbase/stablecoin (USDC or similar) — are the standard payment methods for the project, and Corda verifies payments made through each of them. For card/PayPal the provider signature is the verification. For wire transfer and Zelle the operator verifies the settlement against the provider record, because those channels publish no webhook. For Coinbase/stablecoin the on-chain settlement is verified against the wallet record. In every case the verification result, the provider reference, amount, currency, and UTC verification timestamp are recorded before the transaction is documented within the secure blockchain and ledger, so that the three evidence gates in §11.2.2 can be resolved to the same correlation record.

Approved 2026-08-28 (section 18.4): Zelle processing is the same as PayPal and Coinbase/stablecoin processing. The existing authentication method for each payment service is to be verified by Corda in some manner.

7.3 Sales and Receipt Sequence



8. Mapping and Photogrammetry

8.1 Dedicated Storage

The dedicated local workspace for WebODM and photogrammetry is:

```
/media/scottw/500GBPHOTOGRAM/
```

This drive is authoritative for:

- Incoming drone imagery.
- Validated imagery sets.
- Rejected and quarantined uploads.
- WebODM media and project data.
- NodeODM intermediates.
- Mapping deliverables.
- Mapping processing and provenance manifests.
- pCloud/IPFS archive staging.
- Mapping-database backup exports.

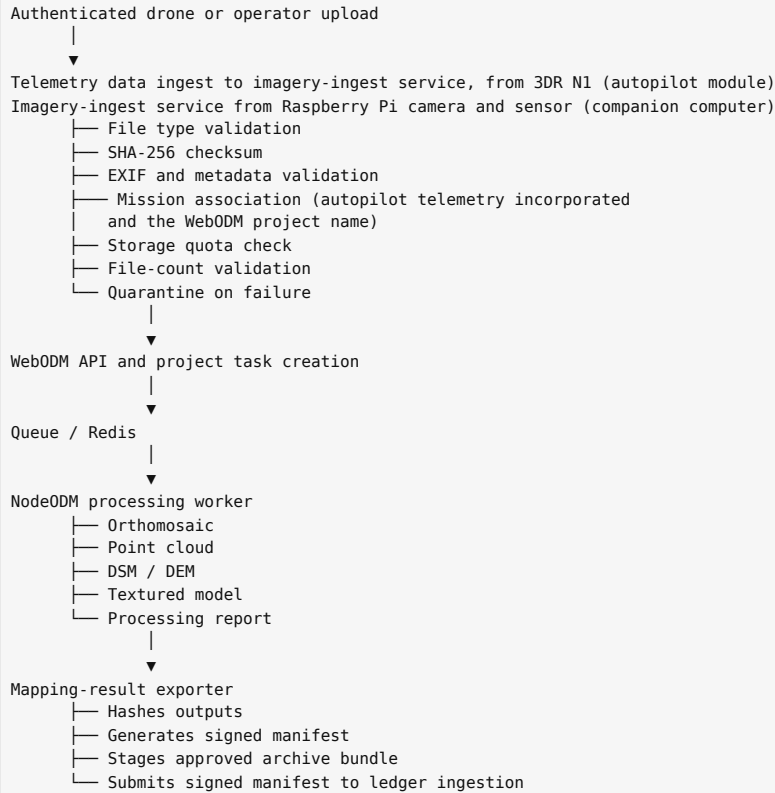
WebODM must not use the root filesystem, `$HOME`, or Podman writable container layers for high-volume processing.

8.2 Required Directory Tree

```
/media/scottw/500GBPHOTOGRAM/
├─ README.md
├─ .mounted-ok
├─ incoming/
│   ├── drone/
│   ├── operator/
│   └─ quarantine/
├─ validated/
│   └─ <mission-id>/
├─ rejected/
│   └─ <mission-id-or-date>/
├─ webodm/
│   ├── media/
│   ├── projects/
│   ├── nodeodm/
│   ├── temp/
│   └─ logs/
├─ deliverables/
│   └─ <mission-id>/
│       ├── orthophoto/
│       ├── point-cloud/
│       ├── dem-dsm/
│       ├── textured-model/
│       ├── reports/
│       └─ manifest/
├─ manifests/
│   ├── intake/
│   ├── processing/
│   └─ ledger-submissions/
├─ exports/
│   ├── pcloud-staging/
│   └─ ipfs-staging/
├─ backups/
│   └─ mapping-db/
├─ retention/
│   ├── pending-review/
│   └─ eligible-for-archive/
└─ tmp/
    └─ processing/
```

No directory in this tree may be world-writable. Use dedicated mapping ownership, explicit groups, and ACLs only when necessary.

8.3 Mapping Processing Flow



8.4 Persistent Locations

Data	Location
Raw images	/media/scottw/500GBPHOTOGRAM/incoming/
Validated images	/media/scottw/500GBPHOTOGRAM/validated/
WebODM media/projects	/media/scottw/500GBPHOTOGRAM/webodm/
Intermediate work	/media/scottw/500GBPHOTOGRAM/webodm/nodeodm/ and tmp/
Deliverables	/media/scottw/500GBPHOTOGRAM/deliverables/
Mapping PostgreSQL	/ALWAYSON/data/mapping/postgres/ or approved Podman volume
Redis persistence	/ALWAYSON/data/mapping/redis/
Signed manifests	/ALWAYSON/artifacts/mapping-manifests/

8.5 Mapping Mount Validation

The drive must be identified by filesystem UUID, not by `/dev/sdX`.

WebODM must refuse to start when:

- The mount is absent.
- The mountpoint resolves to the root filesystem.
- The mounted UUID differs from the approved UUID.
- `.mounted-ok` is absent.
- Available space is below the configured minimum.
- Required directories are missing.
- Mapping service ownership or permissions are incorrect.

Required validation:

```
lsblk -f
findmnt /media/scottw/500GBPHOTOGRAM
blkid
df -hT /media/scottw/500GBPHOTOGRAM
```

Begin with CPU-only validation. Enable GTX 1080 access only after validated container GPU runtime, driver compatibility, measurable workload benefit, and a documented CPU-only recovery path.

8.6 3D Model Identity and Database Cross-Referencing

Every 3D model, model revision, component, assembly, and derived artifact must be addressable from the same database and ledger correlation system used for sales and receipts. The model file is not itself the authority; the authoritative relationship is the PostgreSQL registry entry plus the signed content manifest.

8.6.1 Identifier hierarchy

Use a stable, globally unique `model_object_id` for the logical object and a separate `model_revision_id` for each version:

```
model_object_id    # Stable identity of the logical 3D object or assembly
model_revision_id  # One specific model revision/artifact
serial_number      # Physical asset, when the model represents a sold product
correlation_id     # Business/event correlation across PostgreSQL and Corda
receipt_number     # Commercial receipt, when the model is sold
event_timestamp_utc # When the relationship/event was recorded
content_hash_sha256 # Hash of the exact model file or packaged artifact
```

`model_object_id` remains stable across revisions. A revised model must not reuse an old revision ID. `serial_number` links the model to a physical product; it is not a replacement for the model object ID.

3D objects come and go; the data about them must not. The identity, the registry record, and the ledger references must remain consistent for the life of the object, and must be able to exist in either of two states:

- **With no 3D object at all.** A `model_object_id` may exist with no current revision — for example before any geometry is authored, or after every revision has been retired. The registry row, its links, and its Corda references stay valid and remain the authoritative history.
- **With a connection to a new object.** If a replacement object is created, it receives its own `model_object_id`, and the two are related by an explicit link row of type `supersedes` / `replaced_by` in `model_object_links`. The original identity is never overwritten or reused.

Consequently, no business record may depend on the presence of a 3D file. Serial numbers, receipts, entitlements, manifests, and Corda state reference the `model_object_id` and its revisions, never a file path. `content_hash_sha256` is an attribute of a revision, not of the object, and a revision whose artifact is later removed remains as a `superseded` or `retired` record with its hash intact.

8.6.2 Metadata carried with the 3D model

Each model package must carry a sidecar metadata document or embedded metadata block containing at least:

```
{
  "model_object_id": "OBJ-300X3-BATTERY-0001",
  "model_revision_id": "REV-2026-09-25-01",
  "object_type": "cad_assembly",
  "source_system": "cad_release",
  "serial_number": "SN-300X3-000042",
  "correlation_id": "ORDER-2026-000123-A",
  "receipt_number": "RCPT-2026-000123",
  "event_timestamp_utc": "2026-09-25T12:34:56Z",
  "schema_version": "1.0",
  "content_hash_sha256": "SHA256_DIGEST",
  "source_artifact_reference": "opaque internal reference",
  "license_reference": "approved license/terms reference",
  "is_public_proof_eligible": false
}
```

The metadata is cross-referenced, not duplicated wholesale: the model contains identity and reference fields, PostgreSQL contains the operational record, and Corda contains the signed state/reference.

8.6.3 Database registry

The model registry should be implemented in PostgreSQL with tables equivalent to:

```

model_objects
  model_object_id, object_type, canonical_name, created_at_utc, created_by,
  current_revision_id

model_revisions
  model_revision_id, model_object_id, revision_number, content_hash_sha256,
  source_artifact_reference, archive_reference, license_reference,
  created_at_utc, created_by

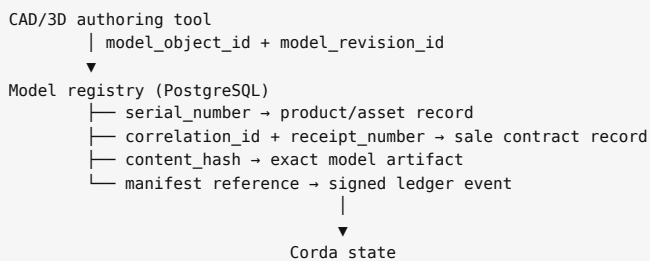
model_object_links
  model_object_id, link_type, serial_number, correlation_id, receipt_number,
  event_timestamp_utc, valid_from_utc, valid_to_utc

model_ledger_references
  model_revision_id, corda_event_type, corda_transaction_id, corda_state,
  corda_confirmed_at_utc, manifest_reference

```

`model_object_links` is the cross-reference table. It relates a model object to a product, serial number, receipt, order, mapping project, simulation result, release, or other approved object without embedding the operational record in the 3D file.

8.6.4 Cross-reference flow



A viewer, CAD tool, WebODM/NodeODM exporter, or marketing application resolves a model by reading its object/revision IDs, validating the content hash, looking up the PostgreSQL registry, following approved links to the serial/receipt record, following the Corda projection, and returning only fields permitted for that audience.

8.6.5 Integrity and relationship rules

- A model revision has exactly one `model_revision_id`.
- A model revision has one immutable `content_hash_sha256`.
- A revision ID cannot point to different content hashes.
- A model object may have many revisions, but only one current revision.
- A physical serial number may have many model revisions over its lifetime.
- A receipt may reference many model objects or serials through the link table.
- A model relationship records `event_timestamp_utc` and its source.
- Superseded revisions are preserved and marked `superseded`, never reused.
- A Corda reference is required before a model is called provenance-verified.
- A content hash proves file integrity, not authenticity or publication safety.

8.6.6 Public and private model metadata

Internal metadata may contain serial numbers, correlation IDs, and opaque references. Public marketing metadata should contain only approved fields:

```

model_object_id or public proof ID
product/SKU
approved serial or proof token
revision label
provenance status
public verification reference
content hash or public proof hash
license/terms reference

```

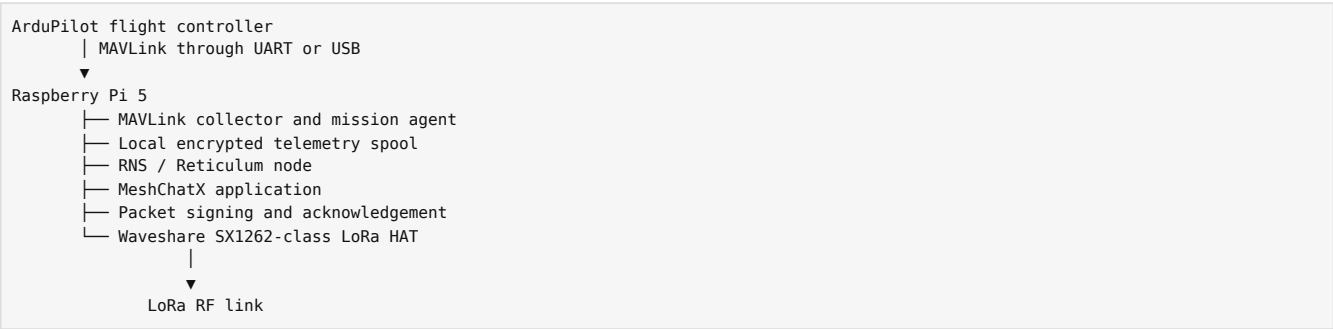
Do not publish customer identity, receipt totals, addresses, payment references, private simulation data, internal paths, or Corda transaction details unless that disclosure is explicitly approved.

8.6.7 Relationship to the sale/receipt process

For a sold product, the 3D model metadata (`model_object_id`, `model_revision_id`, `serial_number`) resolves to the PostgreSQL model registry, `sale_contract_lines`, receipt/correlation projection, and signed Corda provenance reference. The receipt and model may each show a reference to the same correlation record. Neither file is the authoritative sale ledger.

9. Field and LoRa Architecture

9.1 Drone-Side System



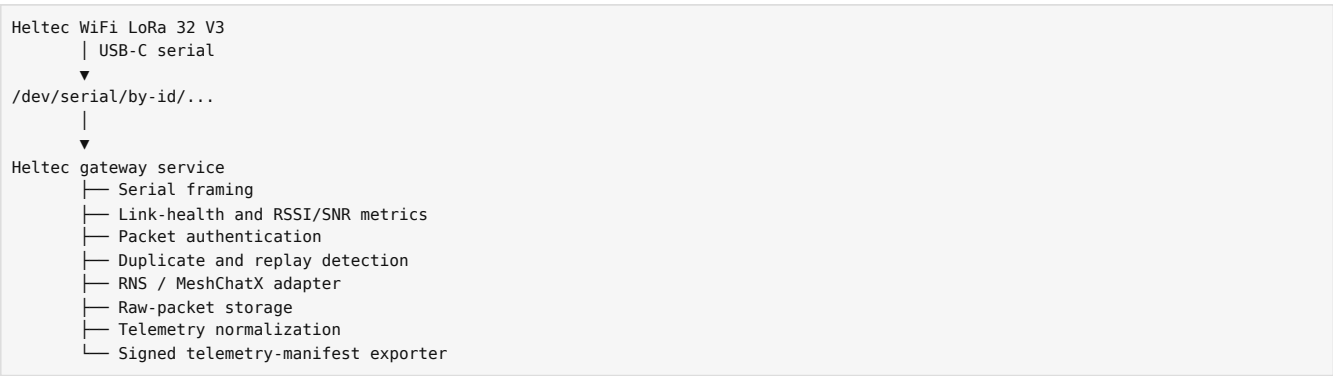
9.2 Desktop Gateway

9.2.1 MeshChatX Local Service Port

The desktop MeshChatX application uses the dedicated loopback port `https://127.0.0.1:18000` for its native backend and local web UI. This port is separate from the ALWAYS ON mapping service listener on `127.0.0.1:8000`.

Service	Domain	Listener	Exposure	Ownership
MeshChatX native backend / web UI	Field / Reticulum	<code>https://127.0.0.1:18000</code>	Loopback only	<code>scottw user service</code>
WebODM web service	Mapping / <code>ao-mapping</code>	<code>127.0.0.1:8000</code>	Loopback only	<code>ao-webodm-web.service</code>
Reticulum transport	Field / Reticulum	Reticulum-configured interfaces	No HTTP listener	Embedded MeshChatX backend

MeshChatX uses its self-signed local certificate; clients must use HTTPS and accept the local certificate. The MeshChatX port is not a public ingress and must not be published through Podman, nginx, Cloudflare, or a router. WebODM and MeshChatX must not share a listener. The desktop launcher and watchdog must use port `18000`; changing one without the others is a configuration error.



The host uses two separate raw-LoRa/Reticulum interfaces. **This is the single table for radio details** — hardware, band, operational state, and the remaining observation for both radios are combined here, so the pair is read in one place rather than from two tables that have to be reconciled.

Radio	Hardware	Configured band	Operational state	Remaining observation
PEOPLE-RADIO	Heltec LoRa 32 V3, SX1262, RNode firmware 1.85	915 MHz	Functional	Characterize feedback observed on this band
DRONE-RADIO	Heltec LoRa 32 V3, SX1262, RNode firmware 1.85	917 MHz	Functional	Characterize feedback observed on this band

Both RNodes are functional and initialize successfully in the active MeshChatX process. This confirms local device detection, serial access, and RNode configuration. RF feedback is observable on both configured bands, as recorded in the table above.

The remaining radio parameters — bandwidth, spreading factor, coding rate, transmit power, and mode — are recorded only in the version-controlled US915 radio profiles in §9.4, not in this README.

The different frequencies and airtimes intentionally separate the public people-facing radio from the private drone/IoT radio. They must not be treated as interchangeable interfaces or combined into one RF channel without an approved frequency plan.

The live host configuration is under `/home/scottw/.reticulum/`. MeshChatX runs headlessly at `127.0.0.1:18000`. Its embedded Reticulum runtime is initialized from `/home/scottw/.reticulum/config`, while MeshChatX identity, repository, and application state are stored under `/home/scottw/.reticulum-meshchatx/`.

Both RNodes are functional and initialize successfully in the active MeshChatX process. This confirms local device detection, serial access, and RNode configuration. RF feedback is observable on both configured bands, as recorded in the table above.

“Feedback” is an operator observation, not yet a diagnosed fault. Potential categories include self-feedback, nearby RF activity, interference, harmonics, spurious transmission, antenna coupling, or reflected energy. Do not change power, frequency, bandwidth, spreading factor, coding rate, antenna, or transmit mode until the source and severity are measured.

Both CP2102 bridges expose the same USB serial descriptor `Silicon_Labs_CP2102_USB_to_UART_Bridge_Controller_0001`. Device identity must therefore be resolved through the stable PCI/USB `by-path` location and the recorded SX1262 MAC address. The USB serial descriptor alone is not a unique radio identity.

9.3 Operational Security

The MeshChatX web interface is restricted to `127.0.0.1:18000`.

The Reticulum `Public Gateway` is configured to listen on `0.0.0.0:4242`. Because this listener binds all local IPv4 interfaces, it is not loopback-restricted. Whether it is reachable from the LAN or Internet depends on firewall and upstream controls, which could not be verified without root privileges during the review. The host had `192.168.87.135/24` on `wlp3s0`, making `192.168.87.135:4242` a potential LAN path unless blocked. This does not prove successful external access. A loopback-only web UI does not make the underlying Reticulum gateway private.

9.4 Radio Profile Requirements

Both radio ends must be verified as compatible US915 hardware variants. Matching SX1262-family radio chips do not guarantee protocol compatibility.

Version-controlled profiles:

```
/ALWAYSON/config/field/heltec-v3/radio-profile-us915.yaml
/ALWAYSON/config/drone/waveshare-lora/radio-profile-us915.yaml
```

The profiles must define identical or explicitly interoperable values for:

- Frequency or channel plan.
- Bandwidth.
- Spreading factor.
- Coding rate.
- Preamble length.
- Transmit power.
- Sync word or network identifier.
- Packet framing.
- Maximum packet size.
- Encryption key identifier.
- Device public identity.
- Sequence number and replay-protection policy.
- Acknowledgement policy.
- Retry and backoff policy.
- Airtime limits.

Do not describe this system as LoRaWAN unless it implements a true LoRaWAN device, gateway, and network-server architecture. The field implementation is primarily an RNode-based Reticulum mesh. The Heltec V3 and Raspberry Pi/Waveshare radios must use matched, approved US915 channel plans. Any separate LoRaWAN or public-discussion service must use different radio bands and settings and remain isolated from the field telemetry mesh.

10. Simulation Architecture

10.1 Vehicle Simulation

```
ao-sim-vehicle
├─ ROS 2 Lyrical
├─ Gazebo Sim 10.5.0
├─ ArduPilot SITL
├─ ROS-Gazebo bridge
├─ MAVLink router
├─ QGroundControl simulation client
├─ Optional Stable-Baselines3 evaluation
├─ Mission and scenario runner
└─ Vehicle-result exporter
```

```
ROS_DOMAIN_ID=21
GZ_PARTITION=alwayson_vehicle_sim
```

10.1.1 SITL method, verified against ardupilot.org

Verified 2026-09-29 against the ArduPilot developer documentation. SITL (Software In The Loop) is the autopilot firmware built as an ordinary native C++ executable and run without any hardware; sensor data comes from the simulator. ArduPilot connects to the simulator over MAVLink, and MAVProxy is the ground control station.

```
Gazebo (gz sim -v4 -r <world>.sdf)
├─ ArduPilot Gazebo system plugin
├─ (github.com/ArduPilot/ardupilot_gazebo – does NOT depend on ROS)
└─
ArduPilot SITL (sim_vehicle.py -v ArduCopter -f gazebo-<model> --model JSON --map --console)
├─ MAVLink
└─
MAVProxy GCS → QGroundControl (simulation client)
```

Required environment, per the ArduPilot documentation:

- GZ_VERSION must be set to the installed Gazebo release.
- GZ_SIM_SYSTEM_PLUGIN_PATH must include the built ardupilot_gazebo plugin directory.
- GZ_SIM_RESOURCE_PATH must include the plugin's models/ and worlds/ directories.
- With ROS 2 in the loop, ArduPilot's DDS support is used: build SITL with DDS enabled, set DDS_ENABLE=1, and make ArduPilot's DDS_DOMAIN_ID match the environment's ROS_DOMAIN_ID. Set them together and relaunch SITL if they change.

The standard example invocations are:

```
gz sim -v4 -r iris_runway.sdf
sim_vehicle.py -v ArduCopter -f gazebo-iris --model JSON --map --console
```

ArduPilot SITL can simulate multi-rotor aircraft, fixed-wing aircraft, ground vehicles, underwater vehicles, camera gimbals, antenna trackers, and a wide variety of optional sensors. The vehicle profiles below therefore must be selectable at run time, ideally at rapid speed, by changing the ArduPilot vehicle type and the Gazebo model, not by rebuilding anything.

Vehicle profiles must allow for switching between the following, ideally at rapid speed, by changing the ArduPilot vehicle type and the Gazebo model:

- Bicopter profile.
- Fixed-wing VTOL tailsitter profile.
- Rover (quadcycle) profile.
- Dual-rotating underwater/submersible profile.
- Boat mast/sail control profile.
- Boat bow/stern thruster profile.
- Boat bow/stern airboat fan profile.
- Wind, terrain, obstacles, routing, takeoff, and landing events.
- Camera, GPS, IMU, barometer, rangefinder, battery, and MAVLink behavior.
- GPS loss, packet loss, actuator faults, sensor drift, and failsafe handling.

Vehicle simulation must never connect to live flight controllers, field radios, real drone telemetry, payment services, customer records, or Corda core.

10.1.2 Baseline capability: 3D world, boning, reinforcement learning objects, HTML portal

Required by ES.1 for **both** simulation domains. For `ao-sim-vehicle` these four are baseline deliverables, not optional extras, and they are tracked as WORK 000800 (ST-07).

3D world setup. The domain stands up its own Gazebo world as a first-class artifact: the world file, its models, their poses, the lighting and environment, the ground and surface materials, the physics and collision configuration, and the spawn points. World setup must be scripted and repeatable from a single entry point rather than assembled by hand in the GUI, so the same world can be rebuilt identically on any host.

Boning. The world carries a boning and alignment structure: a defined datum frame per model, declared mounting and reference surfaces, joint and axis definitions, and the tolerances between them. Boning is what makes the world measurable rather than merely pictorial — a pose is verified against the boning frame instead of being eyeballed, and a model that has drifted out of tolerance is detectable programmatically. The boning data must be exported alongside the world so the same checks run against recorded evidence.

Reinforcement learning objects. The world supplies reinforcement learning objects as the trainable entities for scenario and policy work: marked, individually addressable objects with observable state, reward-relevant properties, and defined reset behaviour. They must be separable from the static world geometry so that a training run can vary object count and placement without rebuilding the world. The optional Stable-Baselines3 evaluation noted above consumes these objects; the objects themselves are required even where no trainer is attached yet.

HTML portal to operation. The whole environment is exposed through a browser-served HTML portal: start, stop, reset, and inspect the world; view the live model and sensor state; run a scenario; select and configure the reinforcement learning objects; and read back boning and tolerance measurements. The portal is the operator surface, so the domain must be fully settable and operable from it without a desktop GUI client. It runs inside `ao-sim-vehicle` and is reachable only by the approved local path — the same isolation rule that forbids this domain from touching live flight controllers applies to the portal as well. It must never become a path by which the simulation reaches anything outside its own domain.

10.2 Fabrication and Facility Simulation

Gazebo Sim model views of the fabrication and facility domain. These show the modelled robot-arm cells, vehicle and shelving layout, and kitchen/storage volumes referenced below. They are **rendered model views, not operational evidence** — no flight-control or live-machinery path is enabled by anything shown here.

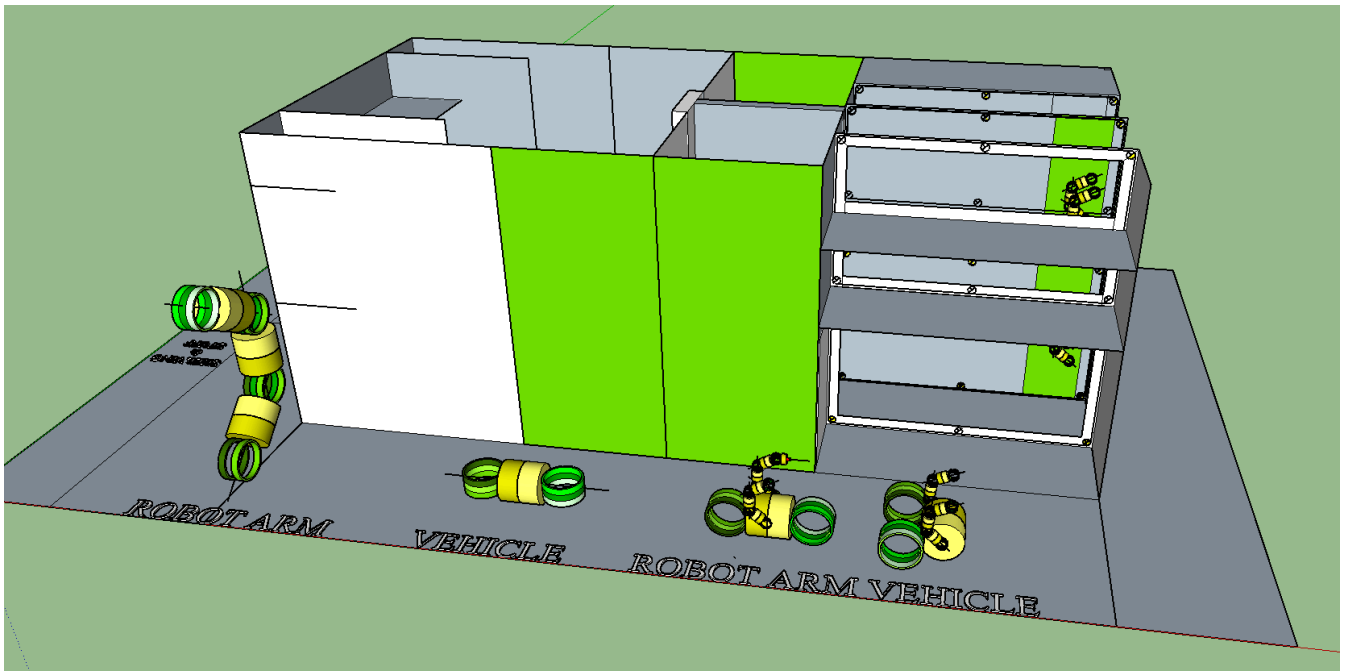


Figure 10.2a — Robot-arm work area, vehicle bay, and robot-arm vehicle bay within the fabrication domain.

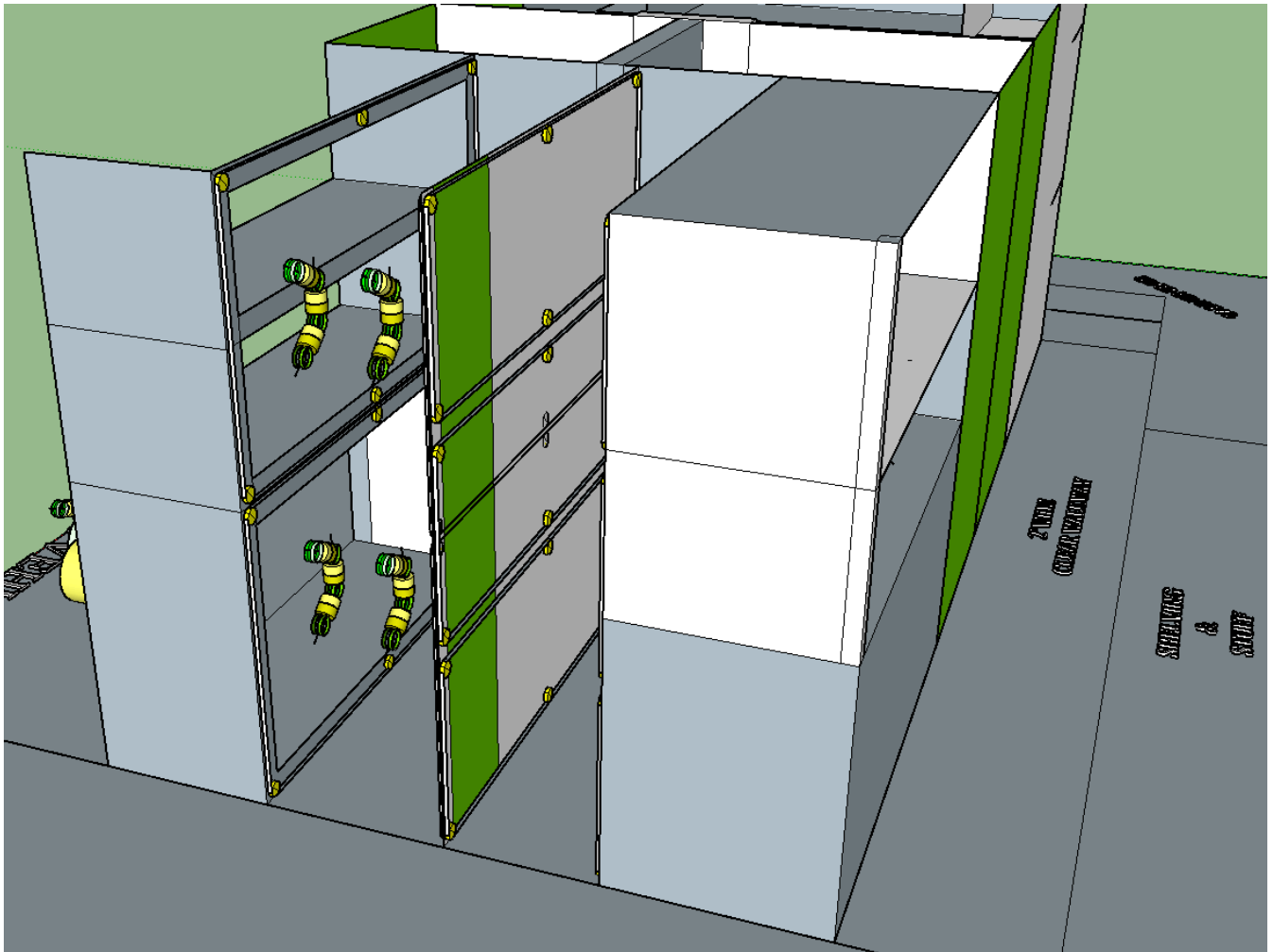


Figure 10.2b — Storage and shelving elevation with robot-arm cells, and the shelving-to-printer aisle.

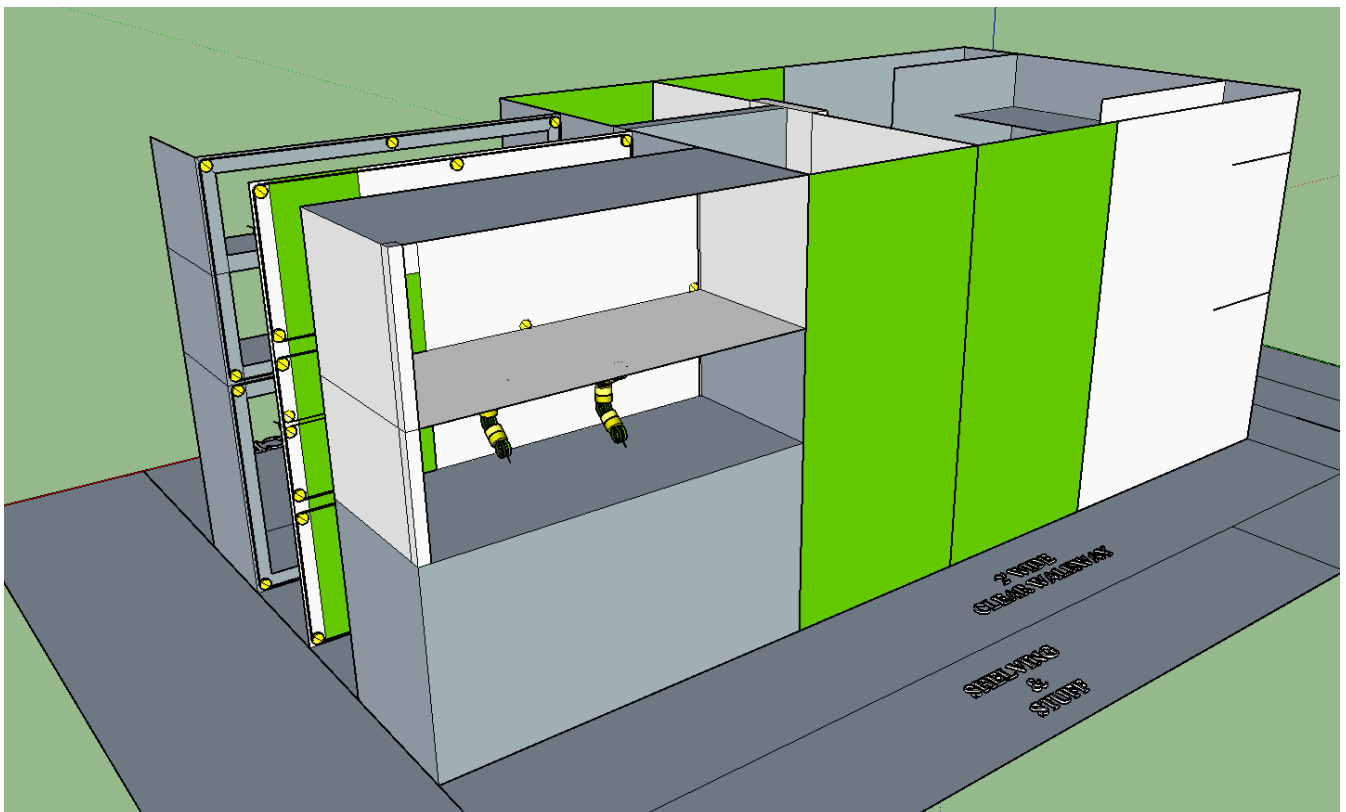


Figure 10.2c — Storage, shelving-to-printer, and kitchen volumes from the reverse approach.

```

ao-sim-fabrication
├─ ROS 2 Lyrical
├─ Gazebo Sim 10.5.0
├─ Industrial engineering and production coordination (receives production data from each machine)
├─ Assembly stations
├─ Storage and inventory cells
├─ Refrigerator, freezer, and pantry models
├─ Kitchen and pass-through models
├─ Carousels and conveyors
├─ Facility scheduler
├─ Safety-zone and interlock model
└─ Fabrication-result exporter

```

```

ROS_DOMAIN_ID=22
GZ_PARTITION=alwayson_fabrication_sim

```

The machines — MainsailOS, Moonraker, Klipper on the BigTreeTech CB1/RPi, and the individual additive-manufacturing machines (3D printers and CNC machines) — are real machines, not simulation nodes. `ao-sim-fabrication` coordinates **all** industrial engineering and production related details, **receiving specific production data from each machine**, and it runs the kitchen.

```

Storage
├─
└─
  Carousel or conveyor
  │
  └─
    Robot-arm pickup
    │
    ├─ 3D printing (individual machine)
    ├─ Additive manufacturing, laser powder bed fusion
    ├─ CNC machining (individual machine)
    ├─ Assembly – production data from each machine
    ├─ Refrigerator or pantry
    └─ Kitchen or pass-through

```

Phase one is simulation only. It must not command live robot arms, 3D printers, CNC machines, laser powder bed fusion systems, refrigeration, carousels, kitchen equipment, or other machinery.

LPBF systems, refrigeration, carousels, kitchen equipment, or other machinery.

Vehicle and fabrication simulation domains require separate:

- Podman networks.
- ROS domain IDs.
- Gazebo partitions.
- DDS configuration.
- Service identities.
- Filesystem mounts.
- Result directories.
- Simulation **manifest-signing** certificates only. These sign exported artifacts; they must not be Corda client identities, and simulation must never open a connection to Corda core (section 10.1).
- Git repositories or clearly separated repository subtrees.
- Artifact manifests.

Use local folder storage under `/ALWAYSON`, in the Gazebo subfolder (verify its exact location with the operator), for SDF, URDF/Xacro, world files, robot definitions, safety zones, task plans, and launch configurations. Use Git LFS or a separate artifact repository for large meshes, textures, point clouds, and generated results.

10.2.1 Baseline capability: 3D world, boning, reinforcement learning objects, HTML portal

Required by ES.1 for **both** simulation domains, and identical in intent to §10.1.2. For `ao-sim-fabrication` these four are baseline deliverables, tracked as WORK 000801 (ST-08). What differs is only what the world contains.

3D world setup. The fabrication world covers the robot-arm cells, the vehicle and shelving layout, and the kitchen and storage volumes shown in Figures 10.2a–10.2c. The world file, models, poses, materials, physics and collision configuration, safety zones, and the printer/CNC and storage footprints must be scripted and repeatable from a single entry point, so the world rebuilds identically on any host.

Boning. This is where boning carries the most weight in this domain, because the modelled cells and machines must line up with the real ones. The world carries a datum frame per cell, declared mounting and reference surfaces for the robot arms and for the printer and CNC beds, the shelving and aisle reference planes, and the joint and axis definitions with their tolerances. The boning data is what lets a simulated reach be checked against the real machine envelope rather than assumed, and it is exported with the world so the same check runs against recorded evidence.

Reinforcement learning objects. The world supplies reinforcement learning objects as the trainable entities for cell and kitchen work: marked parts, stock items, and task targets that are individually addressable, observable, and resettable, kept separate from the static world geometry so a training run can vary object count and placement without rebuilding the world. Robot arms and shuttles are the controlled actors; these objects are what they act on and what reward is measured against.

HTML portal to operation. The environment is exposed through a browser-served HTML portal running inside `ao-sim-fabrication`: start, stop, reset, and inspect the world; view live robot, machine, and stock state; run a cell or kitchen task plan; select and configure the reinforcement learning objects; and read back boning and tolerance measurements. It is the operator surface, so the domain must be fully settable and operable from it without a desktop GUI client, and it is reachable only by the approved local path on `ROS_DOMAIN_ID=22 / GZ_PARTITION=alwayson_fabrication_sim`. It must never become a control path to the real machines: the portal operates the simulation, and the separation from live machinery recorded in §10.2 applies to it exactly as it applies to everything else in this domain.

11. Ledger, Provenance, Archive, and IPFS

11.1 Ledger Authority Policy

Corda is the authoritative ledger for approved business provenance, receipt, entitlement, fulfillment-approval, and release-approval records.

Corda is not the authoritative store for domain-operational source data; that data is stored in the related PostgreSQL database.

Area	Authoritative operational data	Actual network
Sales	Sales PostgreSQL order, fulfillment, and customer-service records	ao-sales
Payment	Verified provider event record and normalized payment state	ao-payment (ingress via ao-ingress-payment)
Field	Raw packet store, telemetry spool, and mission records	ao-field
Mapping	Validated imagery, WebODM project data, processing outputs, and deliverables	ao-mapping
Vehicle simulation	Scenario definitions, run data, and result artifacts	ao-sim-vehicle
Fabrication simulation	Facility/task models, safety scenarios, and result artifacts	ao-sim-fabrication
Archive	Encrypted archive objects and retention records	ao-egress-archive
Ledger	Corda state, PKI, and the complete ledger of debits and credits	ao-ledger-core

The **Area** column above uses plain English names. The `ao-*` names in the far-right **Actual network** column are the real Podman network names and are the ones used everywhere else in this document, in the Quadlet definitions, and in `/ALWAYSON/config/platform/network-cidrs.yaml`. Where an area is served by a controlled adapter rather than a workload network, the adapter is named in the same cell.

Corda records signed references, hashes, approved transitions, and entitlement/provenance data that permit verification without duplicating sensitive or high-volume data.

11.2 Ledger Flow



11.2.1 Cross-System Correlation and Provenance Model

The primary business correlation tuple is:

`serial_number + receipt_number + event_timestamp_utc`

These fields link records across PostgreSQL domains and approved ledger records:

- `serial_number` identifies the physical product, vehicle, component, or asset.

- `receipt_number` identifies the approved commercial transaction or receipt.
- `event_timestamp_utc` identifies when the source event occurred, using ISO-8601 UTC.

The tuple should be accompanied by a source event identifier and schema version:

```
correlation_id
serial_number
receipt_number
event_timestamp_utc
event_type
source_domain
source_record_id
schema_version
content_hash_sha256
```

Example:

```
{
  "correlation_id": "ORDER-2026-000123-LOT-A",
  "serial_number": "SN-300X3-000042",
  "receipt_number": "RCPT-2026-000123",
  "event_timestamp_utc": "2026-09-25T12:34:56.000Z",
  "event_type": "entitlement_issued",
  "source_domain": "sales",
  "source_record_id": "order-line-000123-01",
  "schema_version": "1.0",
  "content_hash_sha256": "SHA256_DIGEST"
}
```

The same correlation fields should be carried into approved records from sales, mapping, field, fulfillment, simulation, and release workflows where the event is relevant. A database view or reporting projection should join the records by the correlation tuple rather than by free-text names or presentation labels.

Ledger responsibility: integrity and provenance, not general encryption

Corda/blockchain records should contain the minimum data needed to verify that an approved event, artifact, receipt, entitlement, or state transition occurred:

```
serial_number
receipt_number
event_timestamp_utc
event_type
state or status
content_hash_sha256
opaque source reference
signature/authorization metadata
```

The ledger should not contain:

```
card numbers, CVV, payment secrets, private keys,
full customer PII, raw telemetry, imagery, point clouds,
or large operational payloads
```

Encryption is performed before sensitive data leaves its authoritative store, for example before pCloud archival replication or private IPFS distribution. Corda then records the encrypted-object reference and content hash. This gives integrity and provenance for the encrypted object without putting the plaintext payload on the ledger.

Sales and marketing reporting flow

```
Sales PostgreSQL salesdb
├── order, product, serial, receipt, payment, fulfillment, entitlement
│   └── correlation tuple:
│       └── serial_number + receipt_number + event_timestamp_utc
├── Metabase (REPORTING – reads salesdb)
│   └── sales/marketing reports and PDF exports
├── Grafana (dashboards and metrics – reads salesdb + Prometheus)
│   └── sales/fulfillment/provenance dashboards
└── signed minimized manifest
    └── Corda receipt/entitlement/provenance state
```

Marketing and sales reporting should use approved PostgreSQL views or projections. The reporting layer may join:

```
product/SKU
serial number
receipt number
order and order-line state
entitlement state
Corda receipt/provenance status
event timestamp
```

It must not infer that a product is fulfilled, entitled, paid, or blockchain-verified solely from a marketing label. Those states must come from the authoritative PostgreSQL event and the approved ledger projection.

Implementation preconditions

Before enabling this flow:

1. Initialize and verify the `salesdb` schema.
2. Define canonical `serial_number`, `receipt_number`, and UTC timestamp fields.
3. Confirm the Metabase reporting identity can read `salesdb` and produce the standard PDF reports and receipts, and confirm the Grafana datasource can read it for dashboards. Neither tool writes into `salesdb`.
4. Define the signed manifest schema and correlation-ID uniqueness rule.
5. Implement ledger-ingest authorization, signature verification, idempotency, replay protection, and audit logging.
6. Complete the Corda key/certificate ceremony.
7. Test the complete correlation path with synthetic data before connecting real sales, payment, customer, or product records.

11.2.2 Mandatory Corda Entry Evidence

Corda entry is blocked until all three evidence classes are present for the same business correlation record:

1. **Sale-request email** - Customer-originated sale/KIT REQUEST email or approved equivalent. - Captures requester, requested items/SKUs, comments, and request timestamp. - Does not by itself prove a contract or payment.
2. **Payment-validation email** - Provider-specific validation for PayPal, Zelle, or Coinbase. - Identifies the provider, provider reference, amount, currency, validation status, and validation timestamp. - Does not by itself prove that funds settled into the approved account.
3. **Funds-transfer verification** - Operator/provider reconciliation evidence that the funds actually transferred and settled. - Records settlement/available state, transfer reference, amount, currency, and verification timestamp. - Must not be treated as verified merely because a payment was initiated.

The three records must resolve to the same:

```
correlation_id
receipt_number
serial_number(s)
event_timestamp_utc
```

Recommended evidence record:

```
evidence_id
evidence_type = sale_request | payment_validation | funds_transfer_verification
provider = website | paypal | zelle | coinbase | bank | manual_reconciliation
source_reference
received_at_utc
validated_by
content_hash_sha256
status = received | validated | rejected | superseded
```

A sale is not eligible for Corda submission unless:

```
sale_request.status = validated
payment_validation.status = validated
funds_transfer_verification.status = validated
```

Corda records references, hashes, states, and operator authorization for these three gates; it must not store raw payment credentials or unrestricted email content. PostgreSQL stores the detailed evidence metadata and reporting projection. Metabase is for reporting and Grafana is for dashboards and metrics; both show the resulting confirmed state and neither performs or waives the verification.

Before the three gates can be satisfied, the operator key/certificate ceremony must be complete and the correlation path must be proven with synthetic data.

11.2.3 Corda-Managed Sale and Receipt Process

The detailed process, state machine, correlation model, and activation gate are maintained in the canonical runbook:

```
/ALWAYSON/docs/runbooks/corda-sale-receipt-process.md
```

The short rule is:

```
KIT REQUEST/inquiry
→ verified payment
→ PostgreSQL provisional projection
→ signed sale-contract manifest
→ ledger-ingest gateway
→ Corda transaction/state
→ PostgreSQL final projection
→ receipt and reporting views
```

A receipt is not final until Corda has returned a confirmed transaction/state reference and the PostgreSQL projection records that reference. The intake, form, schema, and validator artifacts are:

```
/ALWAYSON/data/sales/kit-request-intake/
/ALWAYSON/forms/three-column-corda-sale-receipt-form.html
/ALWAYSON/forms/three-column-corda-sale-receipt-form.pdf
/ALWAYSON/forms/corda-sale-receipt.html
/ALWAYSON/config/sales/sale-receipt.schema.json
/ALWAYSON/config/sales/sale-receipt.example.json
/ALWAYSON/scripts/validation/validate-sale-receipt.sh
```

The form is an internal operator form. It does not write to PostgreSQL, contact Corda, process payments, or create a public proof. Submission must go through the authorized ledger-ingest workflow after operator review.

11.3 Corda Stores and Private Data

Corda may retain approved private transaction data as an encrypted private payload or encrypted attachment. Corda does not make plaintext private data safe merely by being on a ledger: confidentiality depends on encryption, authorized recipients, key management, access policy, and audit controls.

Corda contract state

Corda state should contain the small, shared, verifiable business facts:

```
transaction_id
correlation_id
receipt_number
order_id
serial_number(s)
sku
model_object_id / model_revision_id
payment_provider
payment-validation reference/hash
funds-transfer reference/hash
payment/settlement state
entitlement/fulfillment/delivery state
Corda transaction ID
timestamps
signatures/authorization metadata
```

Encrypted private payload

Approved private data may be encrypted before submission and stored as a private attachment or confidential private-state object:

```
customer identity and contact details
purchase-request email/content
payment-validation email/content
funds-transfer verification content
full receipt and contract
fulfillment, delivery, return, and support records
private 3D model files and attachments
```

The private payload envelope must include:

```
transaction_id
data_classification = PRIVATE
schema_version
encryption algorithm
encryption key identifier
authorized recipients
payload SHA-256
retention policy identifier
created_at_utc
```

The encryption key must be held by the approved KMS/wallet/key-management process and must never be stored in Corda, PostgreSQL, Git, HTML, logs, or the transaction bundle.

Never store in Corda

```
card numbers
CVV
bank credentials
payment-provider secret keys
passwords
OAuth tokens
private keys
TLS private keys
KMS master keys
data-encryption keys
recovery phrases
```

Corda tracks the transaction ID, state, hashes, references, and authorized signatures. PostgreSQL retains the operational/reporting projection keyed by the same transaction ID. Metabase reports the confirmed state and Grafana shows it on a dashboard; neither creates or waives payment verification.

11.4 Corda Does Not Store

Corda may store approved encrypted private transaction data as described in Section 11.3. It must never store plaintext secrets or unencrypted credentials.

Never store:

```
card numbers
CVV
bank credentials
payment-provider secret keys
passwords
OAuth tokens
private keys
TLS private keys
KMS master keys
data-encryption keys
recovery phrases
```

11.5 Manifest Format

```
{
  "object_id": "UUID",
  "object_type": "sales_receipt | telemetry_batch | map_product | vehicle_simulation | fabrication_simulation",
  "origin_domain": "sales | field | mapping | sim_vehicle | sim_fabrication",
  "created_at_utc": "ISO-8601 UTC timestamp",
  "schema_version": "1.0",
  "content_hash_sha256": "HEX_DIGEST",
  "content_size_bytes": 0,
  "local_storage_reference": "opaque internal reference",
  "ipfs_cid": "optional encrypted CID",
  "pcloud_archive_reference": "optional opaque encrypted reference",
  "authorization_policy_id": "policy ID",
  "producer_key_id": "service key ID",
  "signature": "detached signature"
}
```

11.6 pCloud and IPFS Rules

```
Local source data
├──
├── Content hash
├── Signed manifest
├── Corda receipt or approval state
├── Encrypted pCloud archive
└── Private or encrypted IPFS distribution
```

- Local source data remains authoritative.
 - Encrypt before pCloud archival replication unless an explicitly approved equivalent encryption control applies.
 - Do not place private data, PII, payment data, private keys, raw telemetry, sensitive imagery, or proprietary technical designs on public IPFS.
 - For post-sale transfer of sensitive artifacts, use a private IPFS swarm, controlled pinning, or encryption before any public IPFS distribution.
 - Record content hash and CID separately.
 - Post-sale, store only CIDs and encrypted package references in Corda. IPFS is a post-sale marking/transfer mechanism only (ES.1); it is not archive, accounting, or ledger storage, and Corda has no IPFS dependency.
 - Archive replication occurs only through `ao-egress-archive`.
-

12. Host Installation and Configuration

12.1 Installation Journal

Create the journal before installation activity:

```
sudo install -d -m 0750 -o "$USER" -g "$USER" /ALWAYSON/logs/installation
touch /ALWAYSON/logs/installation/agent-install.log
chmod 0640 /ALWAYSON/logs/installation/agent-install.log
```

12.2 Initial Non-Destructive Inventory

Run before installing or changing anything:

```

echo "===== Timestamp ====="
date --iso-8601=seconds

echo "===== Host ====="
hostnamectl

echo "===== OS ====="
cat /etc/os-release

echo "===== Kernel ====="
uname -a

echo "===== CPU / RAM ====="
lscpu
free -h

echo "===== Storage ====="
lsblk -o NAME,SIZE,FSTYPE,FSVER,LABEL,UUID,MOUNTPOINTS
df -hT

echo "===== Photogrammetry Mount ====="
findmnt /media/scottw/500GBPHOTOGRAM || true

echo "===== Podman ====="
command -v podman || true
podman version 2>&1 || true
podman info 2>&1 || true

echo "===== systemd ====="
systemd --version

echo "===== cgroups ====="
stat -fc %T /sys/fs/cgroup

echo "===== GPU ====="
lspci -nnk | grep -A3 -Ei 'VGA|3D|NVIDIA' || true
command -v nvidia-smi && nvidia-smi || true

echo "===== Network ====="
ip -brief address
ss -tulpn
ss -tulpn6

echo "===== Firewall ====="
sudo ufw status verbose 2>&1 || true
sudo nft list ruleset 2>&1 || true

echo "===== Existing systemd services ====="
systemctl --user list-unit-files --type=service 2>&1 || true

echo "===== Existing containers: current user ====="
podman ps -a 2>&1 || true

echo "===== Existing containers: system store ====="
sudo podman ps -a 2>&1 || true

echo "===== Existing Podman networks: current user ====="
podman network ls 2>&1 || true

echo "===== Existing Podman networks: system store ====="
sudo podman network ls 2>&1 || true

echo "===== Reticulum and MeshChatX ====="
command -v rnsd || true
rnsd --version 2>&1 || true
pgrep -a -f 'rnsd|ReticulumMeshChatX' || true
ss -ltnp 2>/dev/null | grep -E '(:18000|:4242)' || true

if [ -f "$HOME/.reticulum/config" ]; then
    echo "--- Reticulum configuration ---"
    grep -nE '^\s*type =|^interface_enabled|enabled =|^target_host =|^target_port =|^port =|^mode =' \
        "$HOME/.reticulum/config"
fi

if [ -d "$HOME/.reticulum-meshchatx/logs" ]; then
    echo "--- Recent MeshChatX errors and warnings ---"
    tail -n 500 "$HOME/.reticulum-meshchatx/logs/meshchatx.log" \
        | grep -Ei 'error|warning|disabled|interface|umsgpack' || true
fi

echo "===== Serial devices ====="

```

```
ls -l /dev/serial/by-id/ 2>&1 || true  
} | tee -a /ALWAYSON/logs/installation/agent-install.log
```

Pause and report if:

- The photogrammetry drive is not mounted.
- The mountpoint is an ordinary root-filesystem directory.
- Mapping storage is below 100 GB free.
- Existing WebODM, Podman, Docker, Corda, PostgreSQL, ROS, Gazebo, or related services conflict.
- NVIDIA driver state is broken.
- cgroups v2 or the selected Podman runtime mode does not work.
- Firewall policy conflicts with intended isolation.
- A proposed service port is already bound.
- Heltec cannot be found through a stable `/dev/serial/by-id/` path.
- MeshChatX or Reticulum is unexpectedly absent after installation approval.
- The Reticulum configuration contains an interface not present in the approved inventory.
- A supposedly enabled interface repeatedly fails without a documented compensating control.
- The Reticulum gateway listener is reachable from an unapproved network.
- MeshChatX reports persistence, cryptographic-state, or repository-integrity errors.

12.3 Host Dependencies

After inventory review and explicit operator approval:

```
sudo apt update  
  
sudo apt install -y \  
podman \  
uidmap \  
slirp4netns \  
fuse-overlayfs \  
containernetworking-plugins \  
nftables \  
ufw \  
git \  
curl \  
jq \  
ca-certificates \  
gnupg \  
openssl \  
restic \  
smartmontools \  
lm-sensors \  
acl \  
python3 \  
python3-venv \  
python3-pip
```

Verify:

```
podman version  
podman info --debug  
systemctl --user status  
loginctl show-user "$USER" -p Linger  
test "$(stat -fc %T /sys/fs/cgroup)" = "cgroup2fs" && echo "cgroups v2 active"  
sudo aa-status || true
```

13. Podman Runtime and Quadlet Policy

13.1 Rootless and System-Level Podman

Ordinary application workloads should use rootless Podman and user-level Quadlet units.

Rootless Quadlet definitions are normally stored in:

```
~/.config/containers/systemd/
```

System-level Quadlet definitions are stored in:

```
/etc/containers/systemd/
```

System-level services are permitted only where a documented host-hardware, GPU, storage, networking, or service-management requirement makes rootless operation unsuitable.

13.2 Approved Deviation: Mixed Podman Stores

Architecture requirement: Rootless Podman is the preferred default for ordinary workloads.

Current implementation: Mapping smoke-test evidence indicates that at least some WebODM operations executed through the system/rootful Podman store. The evidence includes root-owned mapping backup artifacts and system-side container storage. An empty `podman ps -a` result from an operator shell does not mean system-store containers, images, volumes, or networks are absent.

Compensating controls:

- No `--privileged` containers.
- Internal mapping network only.
- Explicit bind mounts limited to approved mapping paths.
- Pinned image digests.
- `systemd` resource limits and restart policy.
- Validated NVIDIA CDI access only where required.
- No direct public listener.
- Backup and restore evidence retained.

Resolution condition: Before production declaration, record the approved steady-state model for every domain: rootless, system-level, or mixed. Record the unit owner, Quadlet location, storage path, network owner, GPU access method, and rationale.

13.3 /ALWAYSON Layout

```
/ALWAYSON/  
├─ README.md  
├─ VERSION  
├─ docs/  
├─ storefront/  
│   ├── source/  
│   ├── build/  
│   ├── releases/  
│   ├── manifests/  
│   └─ pcloud-public-folder/  
├─ quadlet/  
│   ├── networks/  
│   ├── volumes/  
│   ├── sales/  
│   ├── payment/  
│   ├── field/  
│   ├── mapping/  
│   ├── sim-vehicle/  
│   ├── sim-fabrication/  
│   ├── ledger/  
│   ├── archive/  
│   └─ operations/  
├─ config/  
│   ├── platform/  
│   ├── storefront/  
│   ├── sales/  
│   ├── payment/  
│   ├── drone/  
│   ├── field/  
│   ├── mapping/  
│   ├── sim-vehicle/  
│   ├── sim-fabrication/  
│   ├── ledger/  
│   ├── mastodon/  
│   ├── pcloud/  
│   └─ ipfs/  
├─ secrets/  
├─ data/  
├─ artifacts/  
├─ ipfs/  
├─ pcloud/  
├─ backups/  
├─ logs/  
├─ scripts/  
├─ tests/  
└─ tmp/
```

Initialize source control:

```
cd /ALWAYSON  
git init  
git branch -M main  
  
cat > .gitignore <<'EOF'  
secrets/  
data/  
logs/  
tmp/  
backups/  
pcloud/restore-cache/  
EOF  
  
git add .gitignore  
git commit -m "Initialize ALWAYS ON configuration repository"
```

13.4 Quadlet Network Template

```
# /ALWAYSON/quadlet/networks/ao-mapping.network  
[Network]  
NetworkName=ao-mapping  
Driver=bridge  
Internal=true
```

13.5 Quadlet Service Template

```
# /ALWAYSON/quadlet/mapping/webodm-web.container
[Unit]
Description=ALWAYS ON WebODM Web Service
After=network-online.target
Wants=network-online.target

[Container]
Image=REPLACE_WITH_APPROVED_IMAGE_DIGEST
ContainerName=webodm-web
Network=ao-mapping.network
Volume=/media/scottw/500GBPHOTOGRAM/webodm/media:/webodm/app/media:Z
Volume=/ALWAYSON/config/mapping/webodm:/config:ro,Z
NoNewPrivileges=true

[Service]
Restart=on-failure
RestartSec=15
MemoryMax=12G
CPUQuota=600%
TimeoutStartSec=180

[Install]
WantedBy=default.target
```

This is a structural template only. The exact image, API settings, mounts, service name, environment, and GPU configuration must be taken from the tested and approved WebODM version.

14. Secrets, Service Identity, and Version Controls

14.1 Secret Delivery

Use Podman secrets or systemd credentials. Prefer file-based secret delivery rather than environment variables.

Secret	Authorized domain
Sales database password	Sales only
Payment webhook secret	Payment verifier only
Payment-provider API secret	Payment adapter only
Mastodon OAuth credential	Community adapter only
Local AI credential/configuration if required	AI service only
Field radio key	Field only
Drone signing key	Drone device only
Corda certificates and keystores	Ledger core only
Ledger client certificates	One distinct certificate per exporter/domain
pCloud archive credential	Archive adapter only
IPFS private-swarm/pinning credential	Archive adapter only

Example:

```
[Container]
Secret=sales_db_password,target=/run/secrets/db_password,uid=10001,gid=10001,mode=0400
```

KDE Wallet may hold interactive operator credentials, but unattended production services must use systemd credentials, Podman secrets, or approved service-specific secret files. Secret rotation, revocation, expiration, and recovery procedures must be documented before production use.

14.1.1 KDE Wallet Secret Management (Implemented)

KDE Wallet is the operator-side secret and credential store for this host. This subsection records the implemented integration; the Section 14.1 policy above remains authoritative, and the unattended-delivery deviation is tracked in Section 18 (Open Decisions).

Runtime and tooling:

- Wallet daemon: `kwalletd6`, reached on the `org.kde.kwalletd6` D-Bus name. Wallet: `kdewallet`, auto-unlocked with the operator's Plasma login.

Verification pending (operator, host check). The bus name and the `entryList/hasEntry` method names are to be confirmed on the running host before anything depends on them. KDE Wallet remains the secret authority per ES.1 regardless of the outcome; only these access details are unconfirmed. Do not rely on an unverified name in a script — confirm it first, then record the verified values here. - Management CLI: `scripts/ops/kwallet-provision.sh` (`create-folders`, `put`, `get`). Run only from the interactive Plasma session while the wallet is unlocked. - Boot-time delivery: `scripts/operations/fetch-kwallet-secret.sh` runs as a Quadlet `ExecStartPre`, waits for the desktop session and `kwalletd` (max ~60s), reads the required entries, and writes a service-specific `0600` env file under the unit owner's `~/secrets/` for the unit to consume via `--env-file`. Used by `ao-mastodon-db`, `ao-sales-db`, and `ao-webodm-db` (verified at boot; see the installation journal).

This login-gated behaviour is intended, not a defect. Services that consume Wallet secrets start after the operator's Plasma login and are not expected to start unattended before a user has entered the password. The ~60s wait is the bounded startup allowance for that login, not a fallback that must survive a passwordless boot. Auto-login, if ever enabled, is a convenience for the operator and is not a requirement of this design.

Single service account. All services run under the operator's own account. No service requires a separate service-account user; the `alwayson-sales` (UID 993) Mastodon placement described in section 20.0 is legacy and is not a required or intended arrangement. It is recorded in section 20.0 as observed history and should be consolidated back to the operator account.

Wallet layout (folder: purpose):

Folder	Purpose
ALWAYSON	Boot-time delivery entries consumed by Quadlet units
ao-mastodon	Local 300X3 Mastodon application secrets (Section 15.3) and OpenClaw OAuth material
ao-sales, ao-payment, ao-field, ao-mapping, ao-ledger, ao-archive, ao-admin, ao-sim-vehicle, ao-sim-fabrication	Per-domain credential folders matching the Section 14.1 authorized-domain table (provisioned empty 2026-08-31)

Current entry inventory (names only; values never in Git, logs, or docs):

Folder	Entries
ALWAYSON	mastodon-db-password, sales-db-password, webodm-postgres-password
ao-mastodon	mastodon-secret-key-base, mastodon-otp-secret, mastodon-db-password, mastodon-ar-deterministic-key, mastodon-ar-primary-key, mastodon-ar-derivation-salt, mastodon-admin-password, openclaw-bot-client-id, openclaw-bot-client-secret, openclaw-bot-access-token, openclaw-bot-password, roundtrip/roundtrip2 (test artifacts)

Rules:

- Never print, copy, export, or log entry values; confirm presence only Presence checks use the D-Bus `entryList/hasEntry` methods on `org.kde.kwallet6`.
- Entries are named per service and per purpose; domain folders enforce the Section 14.1 authorized-domain boundaries.
- Rotation, revocation, expiration, and recovery procedures must be documented before production use (Section 14.1 requirement).

14.2 Version Matrix

Maintain:

```
/ALWAYSON/config/platform/version-matrix.yaml
```

```
host:
  os_release: ""
  kernel: ""
  systemd: ""
  podman: ""
  quadlet_capability: ""
  netplan: ""
  nftables: ""
  ufw: ""

gpu:
  model: "EVGA NVIDIA GTX 1080"
  nvidia_driver: ""
  container_runtime_integration: ""
  cuda_runtime_image_digest: ""

mapping:
  webodm_image_digest: ""
  nodeodm_image_digest: ""
  postgresql_version: ""
  redis_version: ""
  processing_profiles_commit: ""

simulation:
  ros2_distribution: "lyrical"
  gazebo_release: "10.5.0"
  ardupilot_commit: ""
  qgroundcontrol_version: ""
  sb3_version: ""

field_chat:
  standalone_rnsd_version: "1.4.2"
  standalone_rnsd_executable: "/home/scottw/.local/bin/rnsd"
  standalone_rnsd_running: false
  active_reticulum_runtime: "embedded in MeshChatX native backend"
  active_embedded_rns_version: "unverified"
  reticulum_config: "/home/scottw/.reticulum/config"
  reticulum_config_sha256: "2df6a8d9fc2037d9e316ac910ec1721c3b5b6af5e301a50b0e92656226cc4098"
  meshchatx_launcher_metadata_version: "4.9.1"
  meshchatx_running_version: "unverified"
  meshchatx_executable: "/home/scottw/Applications/meshchatx-native/ReticulumMeshChatX"
  meshchatx_executable_sha256: "4f403e52b0a5722a49d433f23660b14b43a779fb3cc9a5a90b8f150d77f18890"
  meshchatx_manifest_sha256_match: true
  meshchatx_storage: "/home/scottw/.reticulum-meshchatx"
  meshchatx_local_ui: "127.0.0.1:18000"
  reticulum_public_listener: "0.0.0.0:4242"
  repository_cached_artifact: "reticulum_meshchatx-4.8.4-py3-none-any.whl"
  repository_cached_artifact_running: false
  people_radio:
    hardware: "Heltec WiFi LoRa 32 V3 / SX1262"
    rnode_firmware: "1.85"
    frequency_hz: 915000000
    bandwidth_hz: 125000
    spreading_factor: 7
    coding_rate: 5
    txpower_dbm: 17
  drone_radio:
    hardware: "Heltec WiFi LoRa 32 V3 / SX1262"
    rnode_firmware: "1.85"
    frequency_hz: 917000000
    bandwidth_hz: 250000
    spreading_factor: 7
    coding_rate: 5
    txpower_dbm: 17
    mode: "internal"
    discoverable: false

ledger:
  corda_version: ""
  cordapp_hashes: ""
  postgres_version: ""
  certificate_profile_version: ""
```

15. Sales, Mastodon, OpenClaw, and Local AI

15.1 Sales Database

Use a dedicated sales PostgreSQL database with separate roles:

```
salesdb
sales_api_role
sales_migration_role
sales_backup_role
sales_reporting_role
sales_admin_role
```

The desktop metadata reports MeshChatX 4.9.1. The native executable hash matches `backend-manifest.json`, but its running version was not independently established. A `reticulum_meshchatx-4.8.4-py3-none-any.whl` artifact also exists in the local MeshChatX repository-server identity and is not the verified running artifact.

Core tables:

```
customers
customer_contacts
products
product_versions
orders
order_lines
payment_provider_events
payment_references
receipts
fulfillment_events
entitlements
returns
support_cases
audit_events
```

15.1.1 Three-Form Transaction Bundles

Every purchase transaction uses one ALWAYS ON-issued transaction ID and one folder containing three forms:

```
/ALWAYSON/data/sales/transactions/<transaction-id>/
├─ 01-purchase-request.html
├─ 02-payment-confirmation.html
├─ 03-receipt.html
├─ BUNDLE-STATUS.txt
├─ provider-evidence/
│   ├── paypal.*
│   ├── zelle.*
│   └─ coinbase.*
```

Issue a new bundle:

```
/ALWAYSON/scripts/sales/issue-transaction-bundle.sh
```

The issuer creates a unique ID, pre-fills that ID into all three forms, and creates the provider-evidence directory. The three forms are:

1. Purchase request.
2. Payment confirmation, including provider validation and funds-transfer settlement.
3. Corda receipt, including the three evidence references and Corda state.

Validate the bundle structure:

```
/ALWAYSON/scripts/sales/validate-transaction-bundle.sh \
/ALWAYSON/data/sales/transactions/<transaction-id>
```

The same issued ID must appear in all three forms and in the bundle status. It is also required in the Corda sale-receipt event and the PostgreSQL contract projection. Payment validation must distinguish provider validation from funds settlement, and the receipt must record the three validated evidence references before operator handoff to ledger-ingest.

Corda tracks the transaction ID, payment/ledger state, hashes, and approved references. Detailed private data remains in the encrypted PostgreSQL projection keyed by the same transaction ID; Corda does not store full customer records, raw emails, payment credentials, or unrestricted evidence.

15.1.2 Website KIT REQUEST PDF Intake

Website-generated PDF requests are accepted at:

```
/ALWAYSON/data/sales/kit-request-intake/
```

The current 300x3.com KIT REQUEST flow composes an email with:

```
SUBJECT: 300X3-WEBREQUEST-  
NAME: <name>  
EMAIL: <email>  
KIT REQUESTED: <selected kits>  
COMMENTS: <comments>
```

```
THIS IS A REQUEST FOR INFORMATION, NOT A CONTRACT
```

The intake folder separates requests from sales:

inbox/	Original PDFs placed for intake
extracted/	Extracted text
manifests/	Hashes and intake metadata
receipts/	Final receipts/contracts only after Corda confirmation
review/	Human review records
archive/	Preserved processed request PDFs
quarantine/	Invalid, duplicate, or sensitive-pattern PDFs

Run the non-destructive intake script:

```
/ALWAYSON/scripts/sales/intake-kit-request-pdf.sh \  
/ALWAYSON/data/sales/kit-request-intake/inbox/<request>.pdf
```

The script preserves and hashes the PDF, extracts text, creates a review record, and explicitly sets:

```
classification=kit_request_inquiry  
sale_logged=false  
corda_state=NOT_SUBMITTED
```

It never treats a website request as payment, creates an order, or submits a Corda transaction. A verified payment event is required before the PostgreSQL sale projection is created. A final receipt requires a Corda-confirmed transaction/state reference written back to PostgreSQL. Metabase reports and Grafana dashboards read the resulting approved projections; neither creates the sale.

15.2 Community and AI Controls

Mastodon/community controls:

- Dedicated OAuth registration.
- Minimum necessary scopes.
- External access only through the controlled sales-domain egress bridge when explicitly enabled.
- Rate limits.
- Separate approval workflow.
- Immutable publication audit log.
- No payment, field, mapping, simulation, or Corda-core access.

OpenClaw uses the local LM Studio model for support drafting and the deployed `mastodon-openclaw-bridge.service` automatically answers new Mastodon mentions and replies as `bot`. The bridge polls the local Mastodon API every 10 seconds, persists its notification cursor, skips historical notifications and its own posts, and posts threaded public replies locally. It does not publish to any other service. Human approval remains required for pricing, orders, shipping, warranties, financial topics, technical claims, safety guidance, legal statements, and any publication outside the local bridge workflow.

15.3 Local 300X3 Mastodon Deployment

The 300X3 Mastodon instance (Mastodon 4.3.7, containerized in the authoritative `alwayson-sales` rootless Podman store) is publicly federated at <https://mastodon.300x3.com>. The main storefront remains on <https://300x3.com> and <https://www.300x3.com>;

it is not routed to Mastodon. Operators use Konqueror and OpenClaw on the desktop. Tokodon is removed completely (ES.1) and is not a supported client.

Architecture requirements and verified state:

- `ao-sales` remains `Internal=true` and contains the Mastodon database, Redis, streaming service, and web origin. Database and Redis are not attached to the egress network.
- Community publication is carried inside `ao-sales`. The Mastodon web and background-worker containers reach federation delivery through a controlled outbound bridge only; no database, Redis, or streaming container is attached.
- Origin web and streaming remain loopback-only: `127.0.0.1:3000` and `127.0.0.1:4000`.
- The sole public Mastodon entry is the dedicated Cloudflare Tunnel hostname `mastodon.300x3.com`, routed to `127.0.0.1:3000` by `cloudflared-alwayson.service`. The storefront hostnames are excluded from the Mastodon tunnel and retain the filedn redirect behavior.
- Mastodon identity is `LOCAL_DOMAIN=mastodon.300x3.com`. The local user records retain login emails `admin@300x3.com` and `bot@300x3.com`, while their canonical ActivityPub identities are `admin@mastodon.300x3.com` and `bot@mastodon.300x3.com`.
- Public actor and WebFinger endpoints were verified at `https://mastodon.300x3.com/actor` and `https://mastodon.300x3.com/.well-known/webfinger`.
- The tunnel currently uses HTTP/2 transport because QUIC stream timeouts were observed on this host. Local and public health checks returned HTTP 200.
- Open registration remains enabled with the approval gate; approval applies to new account registration, not to following an existing local account.
- No passwords, OAuth secrets, API keys, tunnel credentials, or access tokens are committed to Git or recorded in this README.

15.4 Federation Publication of the Local 300X3 Instance

Category: Implemented and operational (completed 2026-09-24).

Scope. Join the fediverse as the 300X3 instance so that public posts from the local deployment appear on external Mastodon servers, including `mastodon.social`.

Federation is a mutual, inbound-and-outbound protocol: remote servers (including `mastodon.social`) must reach this instance over the public internet using HTTPS, and this instance must be able to deliver outbound activity to remote inboxes. The former loopback-only validation stage (Section 15.3) is superseded by the Cloudflare Tunnel edge. Federation is publicly reachable at `https://mastodon.300x3.com`; the main storefront remains on the `apex/www` hostnames and is not routed to Mastodon.

15.4.1 Architecture Requirements

Identity is verified against the live instance: WebFinger and `/api/v1/instance` both report `mastodon.300x3.com`, while `300x3.com` serves the static storefront. The `scottw` and `alwayson-sales` service accounts are separated so the desktop user cannot start a second Mastodon (Section 20.0). Open configuration drift against these values is tracked in section 19.3.

Area	Architecture requirement
Public instance domain	Dedicated <code>mastodon.300x3.com</code> ; canonical handles are <code>user@mastodon.300x3.com</code> . The storefront hostnames remain separate.
Storefront preservation	<code>300x3.com</code> and <code>www.300x3.com</code> retain the filedn static-site redirect; Mastodon is not deployed under a <code>/mastodon</code> subpath.
TLS	Required at the public edge; Cloudflare terminates TLS for <code>mastodon.300x3.com</code> .
Inbound reachability	Cloudflare Tunnel connector <code>cloudflared-alwayson.service</code> routes only the dedicated hostname to <code>127.0.0.1:3000</code> .
Outbound reachability	<code>mastodon-web</code> and the background workers use the sales-domain egress bridge for federation delivery; database, Redis, and streaming remain isolated on <code>ao-sales</code> .
Isolation	<code>ao-sales</code> remains <code>Internal=true</code> ; no database, Redis, or raw origin listener is publicly exposed.
Secrets	Tunnel credentials and API keys remain in protected runtime secret storage; never in Git or this README.
Operator duties	Registration approval, moderation, reports, and blocklists remain operator responsibilities.
Service-account placement	The 5 Mastodon containers run under <code>alwayson-sales</code> (UID 993) in a separate rootless store and systemd user manager , not under <code>scottw</code> . <code>ao-mastodon-web.service</code> / <code>ao-mastodon-streaming.service</code> are masked in the <code>scottw</code> manager to prevent a duplicate instance.

15.4.2 Domain and Mastodon Identity Configuration

Environment changes applied to the authoritative service-account `mastodon.env` on 2026-09-24:

```
LOCAL_DOMAIN=mastodon.300x3.com
LOCAL_HTTPS=true
RAILS_FORCE_SSL=false # Cloudflare edge terminates public TLS
ALTERNATE_DOMAINS=localhost,127.0.0.1
```

- Login emails remain `admin@300x3.com` and `bot@300x3.com`.
- Canonical ActivityPub identities are `admin@mastodon.300x3.com` and `bot@mastodon.300x3.com`.
- WebFinger and actor JSON were verified through the public federation hostname.
- The main storefront remains on `300x3.com` / `www.300x3.com`.
- No `/mastodon` path deployment is used; the dedicated hostname provides the root paths required by ActivityPub.

15.4.3 Edge, TLS, and Network Path

Implemented path (2026-09-22, operator-approved Cloudflare Tunnel variant; workstation-nginx + Let's Encrypt variant below superseded — see Section 18.5):

```
Remote fediverse servers
  | HTTPS 443
  ▼
Cloudflare edge: mastodon.300x3.com
  | HTTP/2 tunnel (QUIC disabled after observed stream timeouts)
  ▼
cloudflared-alwayson.service
  | 127.0.0.1:3000
  ▼
mastodon-web
mastodon-web + mastodon background workers
  | ao-sales egress bridge
  ▼
Remote ActivityPub/WebFinger endpoints
```

The storefront hostnames are not included in this tunnel ingress. Tunnel credentials remain in protected runtime storage and are never committed.

```
Superseded design (retained for history): workstation nginx
- listen 443 ssl; Let's Encrypt certificate (DNS-01)
- port 80 only as ACME/redirect listener
Not required with the tunnel path: edge TLS is provided by Cloudflare
and the origin stays loopback-only (Section 18.5).
```

TLS requirements:

- TLS is mandatory at the edge for all federation traffic (satisfied by Cloudflare edge termination for the tunnel-routed apex hostname).
- Tunnel credentials and origin certificate are stored 0400 under `~/.cloudflared/` and mirrored to KDE Wallet `ao-mastodon`; never in Git or this README. Revoke by deleting/re-creating the tunnel.
- X-Forwarded-Proto: `https` is supplied by cloudflared so Rails generates HTTPS URLs and Secure cookies (validated: instance JSON reports `streaming_api: wss://300x3.com`).

Outbound delivery path:

- Mastodon background workers deliver public activities to remote inboxes over HTTPS/443. Mastodon runs without Sidekiq or any other external background-job service.
- Egress is restricted to the sales-domain egress bridge (Section 3) with HTTPS as the only approved protocol; no broad network membership.
- Rate and retry behavior are Mastodon defaults; no relay subscription is approved unless explicitly decided.

15.4.4 Federation Enablement Sequence

This is the sequence, not a status report. Whether each step is done or outstanding is recorded once, in **\$19.3** (outstanding items) and **ES.3** (component status ST-13 and ST-14). No status is stated here, because a section does not carry its own status.

1. Dedicated Cloudflare Tunnel `alwayson-mastodon-federation` and DNS route for `mastodon.300x3.com`; storefront hostnames excluded.
2. Cloudflare redirect rule narrowed to exclude `mastodon.300x3.com`; the static storefront redirect unchanged.
3. Mastodon identity set to `LOCAL_DOMAIN=mastodon.300x3.com`; actor, WebFinger, and local actor documents verified.
4. Community publication carried inside `ao-sales`; database, Redis, and streaming remain isolated.
5. Tunnel transport switched to HTTP/2 after QUIC stream timeouts; local and public health checks return HTTP 200.

6. @300x3@mastodon.social resolved; public followers collection confirms both local accounts follow it.
7. Public post fetched; local mention records created and native notification processing repaired.
8. Verify reverse follows using the remote following collection and local incoming relationship tables, then perform a fresh signed ActivityPub round-trip test. Tracked in §19.3 items 13 and 14.
9. Bootstrap discovery: from Konqueror signed in at <https://mastodon.300x3.com>, follow at least one account on mastodon.social. Remote servers do not index this instance until first contact occurs. The storefront host <https://300x3.com> is a static site and is **not** routed to Mastodon. Tracked in §19.3 item 20.
10. Validate public-post delivery to mastodon.social and reply/boost round-trips back to the local instance; then submit 300x3.com to the joinmastodon.org directory (operator-approved). Tracked in §19.3 item 21.

Steps 1 through 7 are complete; see ES.3 ST-13 and ST-14 for the evidence and for what remains on the federation edge.

15.4.5 Operational Boundaries After Enablement

- Only public visibility federates; unlisted, private, and direct do not appear on remote servers' explore pages. The OpenClaw draft-by-default and human-approval controls (Sections 11.2.2 and 15.2) remain mandatory for all public publication.
 - post.sh public-post guard remains the script-level approval gate.
 - Federated deletion is best-effort: remote servers may retain cached copies. Content published under this section must be treated as practically irreversible.
 - Publication audit logging (immutable, Section 15.2) must include the remote-delivery outcome for federated statuses.
-

16. Scripts and Operational Standards

16.1 Scripts Layout

```
/ALWAYSON/scripts/  
├─ bootstrap/  
│   ├── 00-inventory.sh  
│   ├── 01-verify-photogrammetry-mount.sh  
│   ├── 02-install-host-dependencies.sh  
│   ├── 03-create-operational-layout.sh  
│   └─ 04-create-podman-networks.sh  
├─ deploy/  
│   ├── deploy-quadlet-domain.sh  
│   ├── validate-quadlet-domain.sh  
│   ├── enable-domain-services.sh  
│   └─ rollback-domain.sh  
├─ validation/  
│   ├── check-photogrammetry-mount.sh  
│   ├── check-open-ports.sh  
│   ├── check-network-isolation.sh  
│   ├── check-secrets-exposure.sh  
│   ├── check-gpu-runtime.sh  
│   ├── check-ledger-ingest.sh  
│   └─ capture-version-matrix.sh  
├─ mapping/  
├─ radio/  
├─ simulation/  
├─ storefront/  
├─ ledger/  
├─ backup/  
├─ restore/  
└─ maintenance/
```

16.2 Script Standard

Every script begins with:

```
#!/usr/bin/env bash  
set -Eeuo pipefail  
IFS=$'\n\t'
```

Every script must:

- Use absolute paths.
- Validate prerequisites.
- Log in UTC.
- Avoid secrets.
- Support `--dry-run` for external or destructive activity.
- Use locks where concurrent invocation could corrupt data.
- Return meaningful exit codes.
- Avoid `eval`.
- Avoid unexamined `|| true`.
- Validate canonical paths before move or delete activity.
- Verify the photogrammetry mount before mapping activity.
- Refuse to delete outside explicitly approved and validated paths.
- Write an audit entry for operational changes.

16.3 Logs-Journals

All logs and journals are kept in one place:

```
/ALWAYSON/LOGS-JOURNALS/
```

Log / journal	How often it updates	Purpose
installation-journal.log	Appended during every install or change session	The installation journal required by §4.1 rule 11. Records commands, versions, significant output, and failures.
operations-journal.log	Appended on every operational change	The operational journal. Records deploys, enable/disable, restarts, and the outcome of validation scripts.
audit.log	Appended on every audited operation	Immutable audit trail of operational changes and authorization decisions.
backup.log	After every backup run	Records each backup run, repository used, snapshot ID, and success/failure.
restore-test.log	After every restore test	Records the isolated restore test: source backup ID, operator, result, and exceptions.
gpu-runtime-check.log	On each GPU runtime validation	Driver/CDI state and whether GPU access was granted to the workload.
script-runs.log	On every script invocation	Which script ran, its arguments, exit code, and dry-run status.
mastodon-local-proxy.log	While the local proxy runs	Local Mastodon proxy activity and errors.
meshchatx.log	Continuously while MeshChatX runs	MeshChatX application log: interface state, connectivity, and persistence errors.
sim-gz-server.log	While the Gazebo server runs	Headless Gazebo simulation output.
sim-foxford-bridge.log	While the bridge runs	Foxglove bridge output and connection state.
sim-clock-bridge.log	While the clock bridge runs	Simulation clock bridge output.
web-console.log	On console operations	Web console operations and their outcomes.
lmstudio-readme-preset.sha256	On preset change	Checksum of the LM Studio README preset, for drift detection.
gpu-runtime/	On each validation	Directory of GPU runtime validation captures.
backup/	After every backup run	Directory of backup run records and repository metadata.
operations/	On each operational change	Directory of per-operation journals, one file per operation.

Logs are classified per §4.2 and are never a place to record secrets.

17. Backup, Restore, Monitoring, and Completion Criteria

17.1 Backup and Restore Policy

Use a 3-2-1 strategy: three copies, two media types, and one off-host/off-site copy.

Frequency	Required activity
Continuous or 15-minute where enabled	Database WAL/archive strategy for critical recovery objectives
Hourly incremental	Configuration, manifests, sales records, field telemetry, current project data
Daily	PostgreSQL dumps for <code>salesdb</code> , <code>mastodon</code> , <code>webodm</code> , and <code>cordadb</code> when active; Corda backup; mapping manifests; simulation exports; storefront releases
Weekly	Repository integrity check and off-host copy validation
Monthly	Isolated restore test
Quarterly	Full disaster-recovery exercise

Restore testing must:

1. Restore to an isolated test path or test host.
2. Validate database integrity.
3. Recalculate artifact hashes.
4. Compare hashes with stored manifests.
5. Verify associated Corda receipt/manifests where available.
6. Record operator, source backup ID, result, and exceptions.
7. Alert on failure.

17.2 Monitoring

Monitoring runs in `ao-admin`, which has no VPN, no explicit allowlist, and no public exposure. Its only permitted output is the Grafana dashboard and the Metabase reports.

Prometheus is for security only. It operates independently of the other systems to ensure security and to address any problems, and it acts alone and independently: it does not depend on Grafana or Metabase, and Grafana and Metabase do not depend on it for the data they read from the business databases. Grafana presents the dashboards and metrics; Metabase produces the reports.

Monitor at minimum:

Component	Required metrics
Host	CPU, RAM, storage health, disk usage, temperature, GPU state, kernel errors
Podman/systemd	Unit state, restart loops, health, image digest
Mapping	Queue depth, failures, duration, disk space, CPU/GPU use
Field	Packet rate, RSSI, SNR, retries, replay rejections, spool depth, gateway uptime
Sales	Payment-verification failures, receipt failures, orders, API latency
AI/community	Model latency, request count, GPU use, approval queue, OAuth failures
Vehicle simulation	Scenario success, SITL/ROS/Gazebo health, result export
Fabrication simulation	Task state, collision/safety events, result export
Ledger	Corda health, ingest failures, certificate expiry, backup age
Backup	Last success, repository health, restore-test result, queue age

Alerts must cover disk pressure, backup failure, failed restore tests, container restart loops, unexpected listeners, failed payment verification, radio disconnection, WebODM backlog, GPU contention, expired certificates, and denied cross-domain traffic.

17.3 Completion Evidence

No installation or deployment agent may claim completion until it produces:

1. Host inventory report.
 2. Photogrammetry-drive report with mount source, UUID, filesystem, free space, ownership, and permission validation.
 3. Installed package and version matrix.
 4. Rootless and/or system Podman/Quadlet verification.
 5. GPU driver and container-runtime validation.
 6. Podman network list and domain-isolation results.
 7. IPv4 and IPv6 firewall/listening-port report.
 8. WebODM CPU-only smoke-test result using the dedicated drive.
 9. Vehicle-simulation smoke-test result.
 10. Fabrication-simulation smoke-test result.
 11. Heltec stable serial-device detection and LoRa-link test result.
 12. Ledger-ingestion test and Corda receipt result.
 13. Sales receipt-manifest test without payment secrets.
 14. Backup execution result.
 15. At least one isolated restore-test result.
 16. A current list of unresolved blockers, deviations, risks, and actions requiring human approval.
-

18. Approved Deviations and Open Decisions

18.1 Simulation Baseline Deviation

Decision: ROS 2 Lyrical and Gazebo Sim 10.5.0 are the installed baseline.

Status: Approved on 2026-08-24.

Rationale: The installed and smoke-tested environment differs from an earlier Jazzy/Harmonic draft.

Required control: Record versions, image digests, compatibility test results, and any future migration plan in the version matrix.

18.2 Corda Version and Database Placement (CORRECTED)

Decision (ES.1, authoritative): Corda is built on **Corda 5 with PostgreSQL**. The previous **V4 test installation and its database are removed**, and **no data needs to be migrated** from them.

Current implementation: the retired V4 node and its H2 database were removed on 2026-09-28 (recorded as `retired` in `/ALWAYSON/config/platform/version-matrix.yaml`). `cordadb` is provisioned on the host PostgreSQL 18 cluster as a separate logical database with separate roles and backup scope, which is the correct placement.

Root cause of the earlier deviation record: the Corda node was **set up with H2 instead of PostgreSQL**. That was the error. The H2 scaffold is the defect, not the PostgreSQL placement, and the earlier text in this section that described PostgreSQL as the deviation has been corrected accordingly.

Status: Corrected and closed as a placement question. Because no data is being migrated, the remaining work is a clean rebuild of the node on Corda 5 against `cordadb`, not a data migration. Until that is done the node is not production-ready and the operator key/certificate ceremony in section 18.3 still applies.

Required control: Document database roles, host-loopback binding, backup scope, restore procedure, and separation from sales/mapping databases.

18.3 Corda Deployment Blocker

Status: Blocked. MUST BE ADDRESSED BEFORE ADDITIONAL CORDA DEVELOPMENT

Condition: Corda node deployment requires the operator key and certificate ceremony.

Rule: Do not generate, replace, export, or activate production ledger keys without explicit operator approval and recorded ceremony output.

18.4 Payment Provider Decision

Status: Decided 2026-08-28.

Decision: PayPal (hosted checkout, provider-signed webhooks) plus **Zelle** for direct US payments PLUS COINBASE STABLECOIN (USDC), used from an operator-built custom HTML storefront. The storefront HTML will be developed externally (lovable.dev) and linked into this project; it remains static and is served from the pCloud Public Folder.

Controls required before enabling:

- PayPal: hosted checkout only; signature-verified webhook via `ao-ingress-payment` -> `ao-payment` verifier; credentials via §14.1 secret delivery; no PayPal secret material in the repo, logs, or pCloud.
- Zelle: manual reconciliation path only (equivalent to the wire-transfer policy in §7.2): operator-verified receipt, auditable reference record, explicit operator approval per §7.2. Zelle provides no public webhooks/API, so no automated verification is permitted until a documented control exists.
- Storefront: static HTML only; no server-side code in the pCloud Public Folder; all dynamic behavior goes through the payment and community adapters. Evaluate the lovable.dev-produced HTML against §4 data policy and the prohibited-paths list before linking.

The prior provider-evaluation draft is retained at `docs/compliance/payment-provider-evaluation.md` for record.

18.5 Mastodon Federation Edge and Identity Decision

Status: Decided and applied 2026-09-24 (closed). Remaining federation work is tracked in section 19.3.

Decision (operator):

- Serve Mastodon publicly at the dedicated hostname `mastodon.300x3.com` through Cloudflare Tunnel.
- Keep `300x3.com` and `www.300x3.com` on the existing static-site redirect; do not route the main website hostname to Mastodon.
- Edge transport is **HTTP/2** because QUIC stream timeouts were observed on this host. The tunnel service is `cloudflared-alwayson.service`.
- Mastodon identity is `LOCAL_DOMAIN=mastodon.300x3.com`; canonical accounts are `admin@mastodon.300x3.com` and `bot@mastodon.300x3.com`.
- Community publication is carried inside `ao-sales`, attached to the Mastodon web and background-worker containers only; database, Redis, and streaming remain on internal `ao-sales`.
- Origin ports remain loopback-only. TLS terminates at Cloudflare.

Verified state:

- Public actor and WebFinger endpoints return HTTP 200.
- The local web and health endpoints return HTTP 200.
- The remote account `@300x3@mastodon.social` lists both local accounts as followers, confirming local-to-remote follows. Its `following` collection is empty; reverse remote-to-local follows are not yet recorded.
- A public remote post was fetched and its local mention records were created; notification delivery was repaired through Mastodon's native notification service after the asynchronous worker failed to materialize the rows.

Resolution condition: verify reverse follows using the remote account's `following` collection and the local incoming relationship tables. Do not mark the reverse direction complete based only on a local outgoing request.

19. Open Implementation Items

Everything in this document is either a standard (stated in sections 1-18, above) or work that remains to be done. There is no third category. **Standards are not repeated here**, even where a standard has an unmet acceptance test — the unmet test belongs in the table below, the rule itself belongs in its own section.

This is the single list of remaining implementation work. Each row names the standard it serves, so the requirement is never lost. Completed and verified items are not listed; they are recorded as evidence in section 20.

19.1 Blocking — the system is not production-ready without these

#	Item	Standard served	Acceptance criteria	Blocks
1	Corda key/certificate ceremony	§18.3, §11	Operator ceremony performed and output recorded. No production ledger keys generated, replaced, exported, or activated without explicit operator approval.	Ledger core, all §11 flows
2	Corda 5 build on PostgreSQL	ES.1, §18.2	Node built on Corda 5 against cordadb in PostgreSQL 18, with the previous V4 installation and database removed and no data migrated; correlation join by receipt number, serial number, and UTC timestamp proven.	Ledger core
3	Controlled ingress/egress adapters	§5.2	ao-ingress-payment, ao-egress-archive, ao-build-update implemented with destination allowlists, validated TLS, separate credentials, and connection logging. Community publication is carried inside ao-sales.	External payment, archive, community connectivity
4	Unattended secret delivery decision	§14.1, §18	Either migrate mastodon-db, sales-db, and webodm-db to Podman secrets or systemd credentials, or record an approved deviation with compensating controls, before any production declaration.	Production declaration
5	Mapping runtime designation	§13.2	WebODM runtime finally designated rootless, system-level, or mixed, and the mixed-store deviation in §13.2 closed or confirmed.	Mapping production declaration

19.2 Payments, sales, and ledger

| # | Item | Standard served | Acceptance criteria | | 6 | Payment credentials into KDE Wallet ao-payment | §14.1 | Folder provisioned per §14.1.1; entry stored through kwallet-provision.sh; no secret in Git, logs, HTML, or Corda. | | 7 | Payment verifier and normalized event model | §7.2, §7.3 | A test payment event produces a verified normalized record. | | 8 | Sales API and receipt/fulfillment workflow | §7.3, §15.1 | A sales receipt manifest can be generated without exposing sensitive data. | | 9 | salesdb schema initialization | §3.3.1, §15.1 | Live application schema initialized; read-only reporting views defined. | | 10 | Corda ingest accepts only approved signed data | §4.4, §11.2 | Ledger-ingest receives signed, minimized manifests only, with authorization, idempotency, replay defence, and audit. | | 11 | pCloud archive credentials | §11.6, §17.1 | Credentials provisioned into ao-archive; non-destructive encrypted replication test approved and run. **Presence-only checks — never print, copy, or export values.** |

19.3 Community, federation, and local AI

#	Item	Standard served	Acceptance criteria
12	Mastodon configuration drift reconciliation	\$15.4	config/mastodon/instance-policy.yaml, mastodon.env.example, version-matrix.yaml, secrets/mastodon/mastodon.env, and fetch-mastodon-env.sh all reconciled to mastodon.300x3.com. Do this before the next Mastodon restart — the helper emits the superseded apex value unconditionally.
13	Reverse-follow validation	\$15.4.4	Confirmed from the remote following collection and local incoming relationship tables, never inferred from local outgoing state.
14	Fresh signed ActivityPub round trip	\$15.4.4	Run after the notification-worker fix; reply/boost round trip received locally.
15	Remote account approval/rejection record	\$15.4.5	Recorded separately from local account follow state.
16	OpenClaw OAuth and conversation validation	\$15.2	OAuth completes over HTTPS at the federation origin; OpenClaw posts a threaded reply per mention; bridge posts only to the local instance.
17	Mastodon service-account consolidation	\$14.1.1, \$20.0	always-on-sales (UID 993) placement folded back to the operator account. No separate service-account user is intended.
18	300x3.com email routing / MX	\$15.3	Delivery confirmed or formally deferred.
19	Per-modal purchase buttons, HTML-300X3	\$7.1.1	Implemented in the repo and the static export mirrored to the pCloud Public Folder.
20	Bootstrap discovery for remote servers	\$15.4.4 step 9	From Konqueror signed in at https://mastodon.300x3.com, follow at least one account on mastodon.social. Remote servers do not index this instance until first contact occurs. https://300x3.com is a static storefront and is not routed to Mastodon.
21	Public-post delivery, round trips, and directory submission	\$15.4.4 step 10	Public-post delivery to mastodon.social and reply/boost round-trips back to the local instance are validated; then 300x3.com is submitted to the joinmastodon.org directory. Directory submission is an external publication and requires explicit operator approval.

19.4 Field, radio, and simulation

#	Item	Standard served	Acceptance criteria
22	RF characterization on both bands	\$9.4	RSSI, SNR, noise floor, packet loss, retry behaviour, and airtime recorded on both RNodes. Closure is a recorded finding, not a fix — if the interference is benign ambient noise, record that. No corrective action unless measurement shows a real fault.
23	End-to-end field link test	\$9.2, \$9.4	Unicast and broadcast proven over each RF path; fail-safe verified on radio, serial-path, and peer loss; no live flight-control path enabled during testing.
24	Cross-band isolation	\$9.4	915 MHz and 917 MHz isolation measured; interference classified as in-band, adjacent-band, harmonic, or spurious.
25	Reticulum gateway listener review	\$9.3, \$9.4	0.0.0.0:4242 reviewed against field-domain firewall policy; reachability decided rather than left unverified.
26	umsgpack persistence error	\$9.2	Classified, or formally accepted as a historical bounded-ratchet defect with restart-persistence evidence.
27	Gazebo GUI clients and DDS policy	\$10.1, \$10.2	Vehicle and fabrication GUI clients deployed; separate DDS/interface policy decided.
28	/ALWAYS0N Gazebo subfolder	\$10.2	Path confirmed by the operator. Currently recorded as an open decision, not a guess.
29	QGroundControl interactive workflow	\$10.1	Interactive SITL workflow validated end to end.
30	Vehicle 3D world, boning, RL objects, HTML portal	ES.1, \$10.1.2	WORK 000800. World setup scripted and repeatable; boning frame and tolerances measurable and exported; RL objects addressable and resettable; the world fully settable and operable from the browser-served HTML portal.
31	Fabrication 3D world, boning, RL objects, HTML portal	ES.1, \$10.2.1	WORK 000801. As item 31, for ao-sim-fabrication, with cell and machine datum frames and boning checked against the real machine envelopes.

These three items are the outstanding measurement work for the field and radio domain. The radio details themselves — hardware, configured band, operational state, and the observation still outstanding on each band — are recorded once, in the single radio table in §9.2, and are not repeated here. The implementation status for this domain is ST-04, ST-05, ST-22, and ST-07 in ES.3; this table records only what is not yet done.

19.5 Operations, reporting, and documentation

#	Item	Standard served	Acceptance criteria
32	kwalletd6 D-Bus access details	§14.1.1	Confirm the <code>org.kde.kwalletd6</code> bus name and the method names on the running host, then record the verified values in §14.1.1. Wallet authority itself is settled; only the access details are unverified.
33	Metabase persistence and first read-only query	§15.1, §17.2	Provision the Metabase application database (dedicated PostgreSQL database for the Metabase schema, saved questions, dashboards, and subscriptions) and the per-source read-only reporting roles, one per PostgreSQL and MySQL source with no write, DDL, or owner privilege. Then confirm state survives restart and a protected ad-hoc read-only reporting query succeeds with no source writes. The application database must never be written to by a reporting source.
34	Version matrix refresh	§14.2	§14.2 currently lags §20 on several rows. Refresh from the host; the matrix and the evidence table must agree.
35	Version-matrix capture automation	§14.2, §16	<code>scripts/validation/capture-version-matrix.sh</code> documented as the producer, with a stated refresh requirement.
36	apparmor-utils and GPU toolkit packages	§12.3	Install list omits packages that later verification blocks assume exist (<code>aa-status</code> check, CDI/GPU access). Reconcile the install list with the verification steps.
37	Asserting install verification	§12.3	The §12.3 verify block prints values without asserting them, and the cgroup check is silent on failure. Add real assertions.
38	Ledger socket-bridge diagnosis	§17.1	<code>scripts/validation/check-ledger-ingest.sh</code> resolved, or the pending operator-run privileged command executed.
39	Scripts layout completeness	§16.1	Layout is missing the <code>sales/</code> directory and <code>validate-sale-receipt.sh</code> , both referenced elsewhere. Add or repoint them.
40	Restore-test script contract	§17.1	The seven-step restore test is unowned; state that the <code>check-*.sh</code> scripts implement it, or the requirement has no executor.
41	WebODM folder validation	§8.5	Tree, ownership, sentinel, and checks validated; WebODM starts only with required validated storage.
42	GPU scheduling and admission policy	ES.1	LM Studio, SketchUp, Gazebo, and WebODM batch scheduling matches the documented priority order.

|---

20. Current Verification Evidence

Status as of 2026-09-25. Each row records the outcome of a check against the running system, not design intent. Where a component is misleading in the operator surface, the discrepancy is stated.

Item	Evidence	Status — see ES.3
Host inventory	Inventory report completed	ST-01 (see ES.3)
Photogrammetry drive	UUID verified; directory tree created	ST-03 (see ES.3)
Package/version matrix	Captured and refreshed	ST-01 (see ES.3)
GUI boundary matrix (section 19)	config/platform/gui-boundary-matrix.yaml created; 10 entries validated (YAML), covering all Section 6.A scope items	Partial
Rootless Podman and Quadlet	Verified; mixed-store deviation documented	ST-01 (see ES.3)
GPU runtime	Driver/CDI verified; CPU baseline and GPU smoke completed	ST-25 (see ES.3)
Domain network isolation	Internal workload networks and test verified	ST-02 (see ES.3)
Firewall and ports	UFW active; prior :80 and :1716 exposure cleared	ST-01 (see ES.3)
WebODM smoke test	apt-76 ; 76 images; GPU-enabled orthophoto produced	ST-03 (see ES.3)
Vehicle simulation	Headless Gazebo 300-iteration and ROS-Gazebo bridge test	ST-07 (see ES.3)
Fabrication simulation	Headless Gazebo 300-iteration and bridge test	ST-08 (see ES.3)
Heltec/LoRa detection	Heltec V3 connected; stable by-id + /dev/heltec-v3 path, udev rule installed, detect-heltec.sh OK, serial probe received c0-framed packets 2026-08-31; LoRa-link test pending ao-field gateway	ST-04, ST-22 (see ES.3)
Corda receipt	Corda 5.2.2 scaffolded; key ceremony pending	ST-09 (see ES.3)
Sales receipt manifest	Sales DB deployed; provider/API pending	ST-11, ST-12 (see ES.3)
Backup	Encrypted restic snapshot 548d9910 completed; recurring schedule automated 2026-08-31 (restic nightly 03:30 timer, weekly integrity verify Sun 04:30, nightly domain DB dumps 03:00 for mastodon/sales/webodm); verification snapshot 32be2a1c saved	ST-18 (see ES.3)
Restore	File hash validated; database 14/14 tables restored	ST-18 (see ES.3)
Monitoring stack (ao-admin)	Prometheus + node_exporter + Grafana run as scottw Quadlet units on ao-admin. Grafana application state is genuinely PostgreSQL-backed against the host cluster over the /var/run/postgresql socket (/api/health reports database: ok), and Prometheus is its only registered datasource. Both Prometheus targets scrape up	ST-19 (see ES.3)
Metabase reporting (ao-admin)	Working. Metabase runs on the host and serves its login page in the browser, which is the expected operator surface. Operator-confirmed 2026-09-28; this supersedes the earlier "not serving" finding. The earlier record described a containerised ao-metabase instance failing during application-database setup and cycling under Restart=on-failure; that container and that fault are not the service the operator uses	ST-20 (see ES.3)
Mastodon local stack (ao-sales)	All 5 containers run under the alwayson-sales service account in a separate rootless store and systemd user manager (Section 20.0); ao-mastodon-web / ao-mastodon-streaming are masked in the scottw manager as a duplicate guard. Web 127.0.0.1:3000 and streaming 127.0.0.1:4000 verified; /api/v1/instance reports mastodon.300x3.com v4.3.7	ST-13 (see ES.3)
Mastodon federation edge	Dedicated Cloudflare Tunnel alwayson-mastodon-federation for mastodon.300x3.com; HTTP/2 connector active; actor and WebFinger 200; storefront hostnames preserved; LOCAL_DOMAIN=mastodon.300x3.com; canonical accounts admin@mastodon.300x3.com and bot@mastodon.300x3.com; community publication carried inside ao-sales on web/background-workers only; local-to-remote follows confirmed; reverse-follow validation pending	ST-14 (see ES.3)
WebODM operator workflow restart	Stack is rootless (scottw/mapping store); system-store recovery step correctly found no system-store containers — no action needed	ST-03 (see ES.3)
ArduPilot SITL MAVLink	ao-ardupilot-sitl.service flags fixed; HEARTBEAT (sysid 1, QUADROTOR, ArduPilot) validated over tcp:127.0.0.1:5760 via pymavlink	ST-07 (see ES.3)
Heltec firmware	RNode firmware 1.85 recorded via rnodeconf; EEPROM valid; signature unverified (operator signing option)	ST-04 (see ES.3)
Reticulum executable	Standalone RNS 1.4.2 available at /home/scottw/.local/bin/rnsd; active Reticulum runtime is embedded in MeshChatX	ST-05 (see ES.3)
MeshChatX deployment	Native headless backend running since 2026-09-24 11:02 local time; local UI bound to 127.0.0.1:18000; desktop metadata declares 4.9.1	ST-06 (see ES.3)

Item	Evidence	Status — see ES.3
Reticulum interface configuration	29 TCP clients use <code>interface_enabled = true</code> ; two RNodes use <code>interface_enabled = true</code> ; one Backbone uses <code>enabled = yes</code> ; zero explicitly disabled	ST-05 (see ES.3)
Reticulum runtime participation	Logs show auto-connections, peering, announces, and LXMf/Nomad network announcements	ST-05 (see ES.3)
Reticulum connectivity	Startup logs contain timeouts, network-unreachable errors, connection refusals, and reconnect cycles for named and discovered interfaces	ST-05 (see ES.3)
Reticulum public gateway	MeshChatX is bound to <code>0.0.0.0:4242</code> ; the host had <code>192.168.87.135/24</code> on Wi-Fi	ST-05 (see ES.3)
Two-radio Reticulum initialization	Both serial paths exist and MeshChatX logged both RNodes as configured and powered up on 2026-09-24	ST-04 (see ES.3)
RNode band feedback	Functional feedback observed on both 915 MHz and 917 MHz paths	ST-04 (see ES.3)
MeshChatX cryptographic-state persistence	12,364 historical <code>umsgpack</code> errors; error block ends before newer 16:29Z and 16:51Z startup entries	ST-05 (see ES.3)
MeshChatX version provenance	Desktop metadata declares 4.9.1; executable hash matches the local manifest; running version remains unverified; repository cache contains a 4.8.4 wheel	ST-06 (see ES.3)

21. Status References

Review the following before changing the platform:

```
README.md
VERSION
git log
docs/compliance/installation-status.md
/ALWAYSON/
```

The current repository README, local working folder, verification evidence, version matrix, and issue log must be reviewed before beginning new work.

Current implementation references:

```
/home/scottw/.openclaw/openclaw.json
(communit publication bridge – carried inside ao-sales)
quadlet/sales/ao-mastodon-web.container
quadlet/sales/ao-mastodon-background-workers.container
scripts/mastodon/federate-local.sh
/home/scottw/.cloudflared/config.yml
```

Do not add API keys, passwords, tunnel credential JSON, or other secrets to this reference list.

Appendix A. Revision History

Revision	Date	Change
v1	2026-09-28	Consolidated single-file plan.
v2	2026-09-28	Structural and consistency revision. Removed the duplicated title/banner block. Renamed the summary sections ES.1-ES.4 to end the collision with body sections 1-21. Resolved Tokodon references against ES.1. Qualified Grafana as optional. Scoped IPFS to post-sale. Deleted the duplicated prohibited-secret list in 11.4. Numbered the unnumbered headings in 11.2 and repaired the broken ordered lists and the truncated 15.4 status prose. Separated two confusable federation/local-operation identifiers.
v4	2026-09-28	Operator decisions applied, topology consolidated, layout optimized.

The change summary for this revision is recorded once, in **ES.1.1 Change Summary**, and is not repeated here. This appendix is the revision index only.

Implementation status is likewise recorded once, as the single list in **ES.3**.